

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

Nitish Srivastava

Geoffrey Hinton

Alex Krizhevsky

Ilya Sutskever

Ruslan Salakhutdinov

NITISH@CS.TORONTO.EDU

HINTON@CS.TORONTO.EDU

KRIZ@CS.TORONTO.EDU

ILYA@CS.TORONTO.EDU

RSALAKHU@CS.TORONTO.EDU

Department of Computer Science
University of Toronto
10 Kings College Road, Rm 3302
Toronto, Ontario, M5S 3G4, Canada.

Editor: Yoshua Bengio

Abstract

Deep neural nets with a large number of parameters are very powerful machine learning systems. However, overfitting is a serious problem in such networks. Large networks are also slow to use, making it difficult to deal with overfitting by combining the predictions of many different large neural nets at test time. Dropout is a technique for addressing this problem. The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much. During training, dropout samples from an exponential number of different “thinned” networks. At test time, it is easy to approximate the effect of averaging the predictions of all these thinned networks by simply using a single unthinned network that has smaller weights. This significantly reduces overfitting and gives major improvements over other regularization methods. We show that dropout improves the performance of neural networks on supervised learning tasks in vision, speech recognition, document classification and computational biology, obtaining state-of-the-art results on many benchmark data sets.

Keywords: neural networks, regularization, model combination, deep learning

1. Introduction

Deep neural networks contain multiple non-linear hidden layers and this makes them very expressive models that can learn very complicated relationships between their inputs and outputs. With limited training data, however, many of these complicated relationships will be the result of sampling noise, so they will exist in the training set but not in real test data even if it is drawn from the same distribution. This leads to overfitting and many methods have been developed for reducing it. These include stopping the training as soon as performance on a validation set starts to get worse, introducing weight penalties of various kinds such as L1 and L2 regularization and soft weight sharing (Nowlan and Hinton, 1992).

With unlimited computation, the best way to “regularize” a fixed-sized model is to average the predictions of all possible settings of the parameters, weighting each setting by

Dropout: 一种防止神经网络过拟合的简单方法。

尼蒂什·斯里瓦斯塔瓦

杰弗里·辛顿

亚历克斯·克里兹夫斯基

伊利亚·苏茨克维尔

鲁слан·萨拉胡特迪诺夫

NITISH@CS.TORONTO.EDU

HINTON@CS.TORONTO.EDU

KRIZ@CS.TORONTO.EDU

ILYA@CS.TORONTO.EDU

RSALAKHU@CS.TORONTO.EDU

加拿大安大略省多伦多市国王学院路
10号3302室，多伦多大学计算机科学
系，邮编M5S3G4。

编辑：约书亚·本吉奥

摘要

拥有大量参数的深度神经网络是功能极为强大的机器学习系统，但过拟合问题是这类网络面临的一个严峻挑战。此外，大型网络的运算速度较慢，因此在测试阶段难以通过整合多个不同大型神经网络的预测结果来有效解决过拟合问题。丢弃机制正是为了解决这一难题而提出的一种技术。其核心思路是在训练过程中随机删除神经网络中的某些单元及其连接，从而避免这些单元过度协同适应。在训练阶段，丢弃机制会从大量经过简化的网络中随机选取样本；而在测试阶段，只需使用一个权重较小的未简化网络，即可大致实现对这些简化网络预测结果取平均的效果。该方法不仅能显著降低过拟合现象，还能在性能上大幅超越其他正则化方法。我们的研究表明，丢弃机制能够在视觉任务、语音识别、文档分类以及计算生物学领域的有监督学习任务中提升神经网络的性能，并在众多基准数据集上取得业界领先的成果。

关键词：神经网络、正则化、模型组合、深度学习

1. 引言

深度神经网络包含多个非线性的隐藏层，正是这一结构使得它们成为具备强大表达能力的模型，能够学习输入与输出之间极为复杂的关系。然而，在训练数据有限的情况下，许多这类复杂关系实际上只是采样噪声的产物，因此虽然它们存在于训练集中，但即便测试集来自相同的分布，也并不会出现在真实的测试数据中。这种情况就会引发过拟合问题，为此人们开发出了多种降低过拟合的方法，包括在验证集上的性能开始下降时就停止训练，采用L1和L2正则化等各种权重惩罚机制，以及软权重共享技术（Nowlan和Hinton，1992）。

在拥有无限计算资源的前提下，对固定尺寸模型进行‘正则化’的最佳方式，就是对所有可能的参数组合所产生的预测结果取平均值，并根据每个参数组合在训练数据下的后验概率来为它们赋予不同的权重。

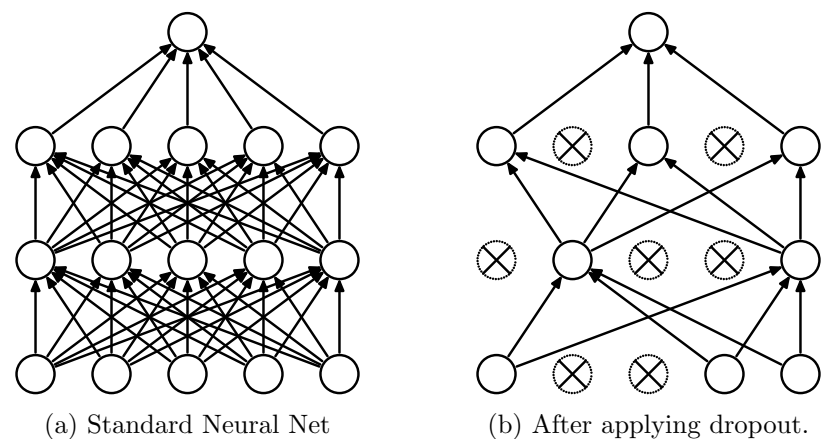


Figure 1: Dropout Neural Net Model. **Left:** A standard neural net with 2 hidden layers. **Right:** An example of a thinned net produced by applying dropout to the network on the left. Crossed units have been dropped.

its posterior probability given the training data. This can sometimes be approximated quite well for simple or small models (Xiong et al., 2011; Salakhutdinov and Mnih, 2008), but we would like to approach the performance of the Bayesian gold standard using considerably less computation. We propose to do this by approximating an equally weighted geometric mean of the predictions of an exponential number of learned models that share parameters.

Model combination nearly always improves the performance of machine learning methods. With large neural networks, however, the obvious idea of averaging the outputs of many separately trained nets is prohibitively expensive. Combining several models is most helpful when the individual models are different from each other and in order to make neural net models different, they should either have different architectures or be trained on different data. Training many different architectures is hard because finding optimal hyperparameters for each architecture is a daunting task and training each large network requires a lot of computation. Moreover, large networks normally require large amounts of training data and there may not be enough data available to train different networks on different subsets of the data. Even if one was able to train many different large networks, using them all at test time is infeasible in applications where it is important to respond quickly.

Dropout is a technique that addresses both these issues. It prevents overfitting and provides a way of approximately combining exponentially many different neural network architectures efficiently. The term “dropout” refers to dropping out units (hidden and visible) in a neural network. By dropping a unit out, we mean temporarily removing it from the network, along with all its incoming and outgoing connections, as shown in Figure 1. The choice of which units to drop is random. In the simplest case, each unit is retained with a fixed probability p independent of other units, where p can be chosen using a validation set or can simply be set at 0.5, which seems to be close to optimal for a wide range of networks and tasks. For the input units, however, the optimal probability of retention is usually closer to 1 than to 0.5.

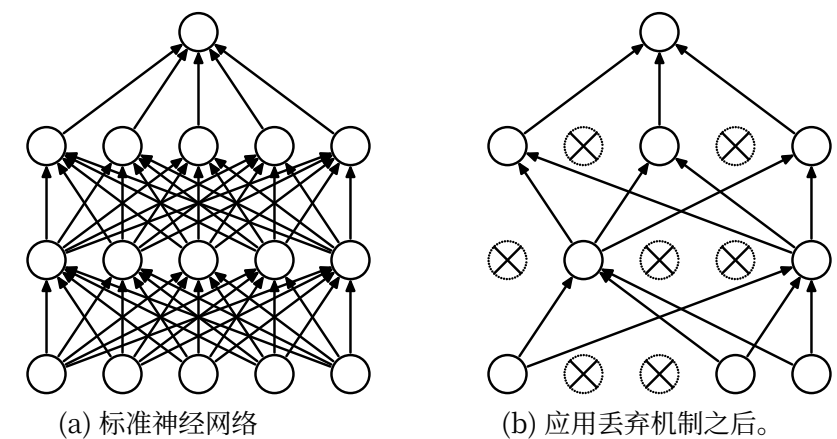


图1: 采用丢弃机制的神经网络模型。左侧展示的是拥有2个隐藏层的标准神经网络；右侧则展示了通过对左侧网络应用丢弃机制后得到的简化网络示例——那些被标记的单元已被舍弃。

对于结构简单或规模较小的模型，这种方法有时能够实现相当准确的近似（参见熊等人，2011；萨拉胡特丁诺夫与Mnih，2008）。不过，我们希望以少得多的计算量就能达到贝叶斯方法的标准基准那样的性能。为此，我们提出采用一种新的方案：对大量共享相同参数的已训练模型所给出的预测结果，先计算出其几何平均数，再对所有预测结果赋予相同的权重，以此作为最终输出。

模型组合几乎总能提升机器学习方法的性能。不过对于规模庞大的神经网络而言，通过平均多个独立训练所得网络的输出这一看似简单的办法，其计算成本实在过高，难以实施。只有当各个模型之间存在差异时，模型组合才能发挥最大作用。为使神经网络模型具有差异性，这些模型要么拥有不同的网络架构，要么在不同的数据集上训练。而训练多种不同的架构难度极大，因为需要为每种架构寻找最优的超参数，且每个大型网络的训练都需要大量计算资源。此外，大型网络通常需要大量的训练数据，可能无法获得足够的数据来让不同的网络在各自的数据子集上训练。即便能够训练出许多不同的大型网络，在那些对响应速度要求较高的应用场景中，也在测试阶段同时使用所有这些网络是不现实的。

丢弃机制是一种能够同时解决上述两个问题的技术。它不仅能有效防止过拟合，还能提供一种高效的方法，将数量呈指数级增长各类不同神经网络网络架构进行整合。所谓“丢弃机制”，指的是在神经网络中随机移除某些单元（包括隐藏层和输出层的单元）。所谓“移除一个单元”，是指如图1所示，暂时将该单元及其所有的输入和输出连接一起从网络中删除。决定移除哪些单元是随机的。在最简单的情况下，每个单元都会以一个固定的概率被保留下来，该概率 p 与其他单元无关；而 p 的值可以通过验证集来确定，或者直接设为0.5，这一数值对于大多数网络和任务而言都接近最优值。不过对于输入层单元来说，其最优的保留概率通常更接近1而非0.5。

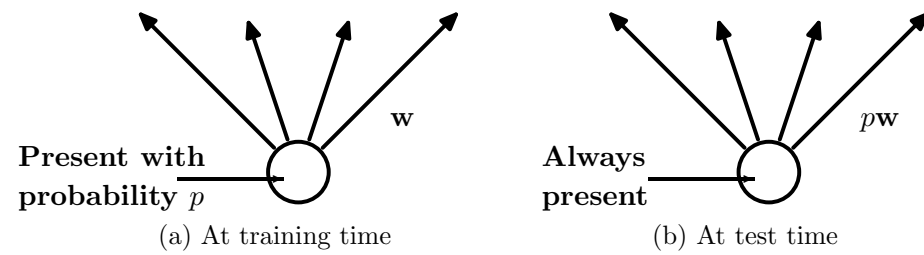


Figure 2: **Left:** A unit at training time that is present with probability p and is connected to units in the next layer with weights $\mathbf{w}$. **Right:** At test time, the unit is always present and the weights are multiplied by p . The output at test time is same as the expected output at training time.

Applying dropout to a neural network amounts to sampling a “thinned” network from it. The thinned network consists of all the units that survived dropout (Figure 1b). A neural net with n units, can be seen as a collection of 2^n possible thinned neural networks. These networks all share weights so that the total number of parameters is still $O(n^2)$, or less. For each presentation of each training case, a new thinned network is sampled and trained. So training a neural network with dropout can be seen as training a collection of 2^n thinned networks with extensive weight sharing, where each thinned network gets trained very rarely, if at all.

At test time, it is not feasible to explicitly average the predictions from exponentially many thinned models. However, a very simple approximate averaging method works well in practice. The idea is to use a single neural net at test time without dropout. The weights of this network are scaled-down versions of the trained weights. If a unit is retained with probability p during training, the outgoing weights of that unit are multiplied by p at test time as shown in Figure 2. This ensures that for any hidden unit the *expected* output (under the distribution used to drop units at training time) is the same as the actual output at test time. By doing this scaling, 2^n networks with shared weights can be combined into a single neural network to be used at test time. We found that training a network with dropout and using this approximate averaging method at test time leads to significantly lower generalization error on a wide variety of classification problems compared to training with other regularization methods.

The idea of dropout is not limited to feed-forward neural nets. It can be more generally applied to graphical models such as Boltzmann Machines. In this paper, we introduce the dropout Restricted Boltzmann Machine model and compare it to standard Restricted Boltzmann Machines (RBM). Our experiments show that dropout RBMs are better than standard RBMs in certain respects.

This paper is structured as follows. Section 2 describes the motivation for this idea. Section 3 describes relevant previous work. Section 4 formally describes the dropout model. Section 5 gives an algorithm for training dropout networks. In Section 6, we present our experimental results where we apply dropout to problems in different domains and compare it with other forms of regularization and model combination. Section 7 analyzes the effect of dropout on different properties of a neural network and describes how dropout interacts with the network’s hyperparameters. Section 8 describes the Dropout RBM model. In Section 9 we explore the idea of marginalizing dropout. In Appendix A we present a practical guide

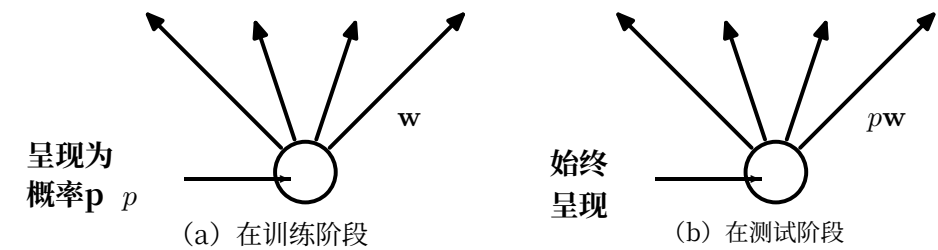


图2: **左侧:** 在训练阶段, 某个单元以 p 的概率存在, 并通过权重 $\mathbf{w}$ 与下一层的单元相连。**右侧:** 在测试阶段, 该单元始终存在, 且其权重会乘以 p 。此时的输出与训练阶段预期的输出一致。

将丢弃机制应用于神经网络, 实际上就是从该网络中抽取出一个“简化网络”。这个简化网络由所有在丢弃过程中存活下来的单元组成 (见图1b)。一个拥有 n 个单元的神经网络, 可以被视作由 2^n 种可能的简化神经网络所构成的集合。这些网络会共享权重, 因此其参数的总数依然保持在 $O(n^2)$ 或更少。在处理每一个训练案例的每一次数据时, 都会抽取并训练一个新的简化网络。由此可见, 使用丢弃机制训练神经网络, 等同于训练由 2^n 个具有广泛权重共享特性的简化网络构成的集合, 而且其中的每一个简化网络几乎都不会被频繁训练, 甚至可能根本不会被训练。

在测试阶段, 要明确地对数量呈指数级增长的经过简化后的模型所给出的预测结果进行平均是不现实的。不过, 一种非常简单的近似平均法在实际应用中效果相当不错。其思路是在测试阶段使用一个不采用丢弃机制的单一神经网络, 该网络的权重为训练所得权重的缩小版本。如果某个单元在训练过程中以 p 的概率被保留下来, 那么如图2所示, 该单元在测试阶段的输出权重会乘以 p 。这样一来, 对于任何一个隐藏单元而言, 在训练阶段用于决定哪些单元被丢弃的相应分布下预期的输出, 都会与测试阶段的实际输出保持一致。通过这种权重缩放的方式, 那些拥有共享权重的 2^n 网络可以被整合为一个单一的神经网络, 以便在测试阶段使用。我们的研究表明, 与采用其他正则化方法进行训练相比, 先训练带有丢弃机制的网络, 再在测试阶段运用这种近似平均法, 能够在多种分类问题上显著降低泛化误差。

丢弃机制这一概念并不仅限于前馈神经网络, 它还可以更广泛地应用于诸如玻尔兹曼机之类的图模型中。在本文中, 我们提出了基于丢弃机制的受限玻尔兹曼机模型, 并将其与标准受限玻尔兹曼机进行对比。实验结果表明, 在某些方面, 采用丢弃机制的受限玻尔兹曼机优于标准受限玻尔兹曼机。

本文的结构如下: 第2节阐述提出该方法的动机, 第3节介绍相关的现有研究, 第4节对Dropout模型进行正式描述, 第5节给出了训练Dropout网络的算法。第6节展示了我们的实验结果, 我们在不同领域的任务中应用了丢弃机制, 并将其与其他形式的正则化方法及模型组合进行了对比。第7节分析了丢弃机制对神经网络各种特性的影响, 同时阐述了该机制与网络超参数之间的相互作用。第8节介绍了带丢弃机制的受限玻尔兹曼机模型。第9节探讨了将丢弃机制边缘化的思路。在附录A中, 我们提供了实用指南。

for training dropout nets. This includes a detailed analysis of the practical considerations involved in choosing hyperparameters when training dropout networks.

2. Motivation

A motivation for dropout comes from a theory of the role of sex in evolution (Livnat et al., 2010). Sexual reproduction involves taking half the genes of one parent and half of the other, adding a very small amount of random mutation, and combining them to produce an offspring. The asexual alternative is to create an offspring with a slightly mutated copy of the parent’s genes. It seems plausible that asexual reproduction should be a better way to optimize individual fitness because a good set of genes that have come to work well together can be passed on directly to the offspring. On the other hand, sexual reproduction is likely to break up these co-adapted sets of genes, especially if these sets are large and, intuitively, this should decrease the fitness of organisms that have already evolved complicated co-adaptations. However, sexual reproduction is the way most advanced organisms evolved.

One possible explanation for the superiority of sexual reproduction is that, over the long term, the criterion for natural selection may not be individual fitness but rather mix-ability of genes. The ability of a set of genes to be able to work well with another random set of genes makes them more robust. Since a gene cannot rely on a large set of partners to be present at all times, it must learn to do something useful on its own or in collaboration with a *small* number of other genes. According to this theory, the role of sexual reproduction is not just to allow useful new genes to spread throughout the population, but also to facilitate this process by reducing complex co-adaptations that would reduce the chance of a new gene improving the fitness of an individual. Similarly, each hidden unit in a neural network trained with dropout must learn to work with a randomly chosen sample of other units. This should make each hidden unit more robust and drive it towards creating useful features on its own without relying on other hidden units to correct its mistakes. However, the hidden units within a layer will still learn to do different things from each other. One might imagine that the net would become robust against dropout by making many copies of each hidden unit, but this is a poor solution for exactly the same reason as replica codes are a poor way to deal with a noisy channel.

A closely related, but slightly different motivation for dropout comes from thinking about successful conspiracies. Ten conspiracies each involving five people is probably a better way to create havoc than one big conspiracy that requires fifty people to all play their parts correctly. If conditions do not change and there is plenty of time for rehearsal, a big conspiracy can work well, but with non-stationary conditions, the smaller the conspiracy the greater its chance of still working. Complex co-adaptations can be trained to work well on a training set, but on novel test data they are far more likely to fail than multiple simpler co-adaptations that achieve the same thing.

3. Related Work

Dropout can be interpreted as a way of regularizing a neural network by adding noise to its hidden units. The idea of adding noise to the states of units has previously been used in the context of Denoising Autoencoders (DAEs) by Vincent et al. (2008, 2010) where noise

these content also include detailed analysis of the practical considerations involved in choosing hyperparameters when training dropout networks.

2. 动机

丢弃机制的提出源自关于性别在进化中作用的理论（Livnat等人，2010年）。有性生殖意味着从一方父母那里获取一半基因，从另一方获取另一半基因，再加入极少量的随机突变，最终将这些基因组合起来产生后代。而无性生殖则是生成一个带有父母基因轻微变异副本的后代。从直觉上看，由于那些已经协同发挥作用的一组优质基因可以直接传递给后代，无性生殖似乎更有利于提升个体的适应能力。然而，有性生殖很可能会打破这些协同进化的基因组合，尤其是当这些基因组合规模较大时；直观上，这应当会降低那些已经演化出复杂协同适应特征的生物的适应能力。尽管如此，有性生殖依然是大多数高级生物进化的途径。

性繁殖之所以更具优势，一种可能的解释是：从长远来看，自然选择的标准或许并非个体的适应能力，而是基因间的兼容性。一组基因若能与另一组随机基因良好协作，就能提升整体的稳定性。由于基因无法始终依赖大量伙伴的存在，它必须学会独立行事，或与少量其他基因协同工作以完成有用功能。根据这一理论，性繁殖的作用不仅在于让有用的新基因在种群中传播，还在于通过减少那些会降低新基因提升个体适应能力的复杂协同适应现象，来促进这一过程。类似地，采用丢弃机制训练的神经网络中的每个隐藏单元，也必须学会与其他随机选取的单元共同协作。这样一来，每个隐藏单元都能变得更加稳定，并且能够在不依赖其他隐藏单元纠正错误的情况下，自主生成有用的特征。不过，同一层内的不同隐藏单元仍会学习执行不同的功能。有人可能会认为，通过复制每个隐藏单元的多份副本，网络就能对丢弃机制具备更强的抵抗力，但这种做法效果不佳，其原因与复制码无法有效应对噪声信道如出一辙。

实际上，另一种与丢弃机制密切相关但略有不同的设计思路源自对成功阴谋活动的思考。相比需要五十人各司其职才能实施的庞大阴谋，由十组各五人组成的阴谋更有可能制造混乱。在条件不变且有充足时间进行准备的情况下，大型阴谋或许能够顺利实施，但在环境不断变化的情况下，规模较小的阴谋反而更有机会依然奏效。复杂的协同适应机制虽然可以在训练集上表现良好，但在面对全新的测试数据时，它们失败的概率远高于那些虽功能相似但结构更为简单的协同适应机制。

3. 相关工作

丢弃机制可以被理解为一种通过对神经网络的隐藏单元添加噪声来对其进行正则化的方法。在 Vincent等人（2008年、2010年）研究去噪自编码器（DAEs）的背景下，早已出现过在单元状态中添加噪声的思路，当时这种做法正是用于

is added to the input units of an autoencoder and the network is trained to reconstruct the noise-free input. Our work extends this idea by showing that dropout can be effectively applied in the hidden layers as well and that it can be interpreted as a form of model averaging. We also show that adding noise is not only useful for unsupervised feature learning but can also be extended to supervised learning problems. In fact, our method can be applied to other neuron-based architectures, for example, Boltzmann Machines. While 5% noise typically works best for DAEs, we found that our weight scaling procedure applied at test time enables us to use much higher noise levels. Dropping out 20% of the input units and 50% of the hidden units was often found to be optimal.

Since dropout can be seen as a *stochastic* regularization technique, it is natural to consider its *deterministic* counterpart which is obtained by marginalizing out the noise. In this paper, we show that, in simple cases, dropout can be analytically marginalized out to obtain deterministic regularization methods. Recently, van der Maaten et al. (2013) also explored deterministic regularizers corresponding to different exponential-family noise distributions, including dropout (which they refer to as “blankout noise”). However, they apply noise to the inputs and only explore models with no hidden layers. Wang and Manning (2013) proposed a method for speeding up dropout by marginalizing dropout noise. Chen et al. (2012) explored marginalization in the context of denoising autoencoders.

In dropout, we minimize the loss function stochastically under a noise distribution. This can be seen as minimizing an expected loss function. Previous work of Globerson and Roweis (2006); Dekel et al. (2010) explored an alternate setting where the loss is minimized when an adversary gets to pick which units to drop. Here, instead of a noise distribution, the maximum number of units that can be dropped is fixed. However, this work also does not explore models with hidden units.

4. Model Description

This section describes the dropout neural network model. Consider a neural network with L hidden layers. Let $l \in \{1, \dots, L\}$ index the hidden layers of the network. Let $\mathbf{z}^{(l)}$ denote the vector of inputs into layer l , $\mathbf{y}^{(l)}$ denote the vector of outputs from layer l ($\mathbf{y}^{(0)} = \mathbf{x}$ is the input). $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the weights and biases at layer l . The feed-forward operation of a standard neural network (Figure 3a) can be described as (for $l \in \{0, \dots, L-1\}$ and any hidden unit i)

$$\begin{aligned} z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)}, \\ y_i^{(l+1)} &= f(z_i^{(l+1)}), \end{aligned}$$

where f is any activation function, for example, $f(x) = 1/(1 + \exp(-x))$.

With dropout, the feed-forward operation becomes (Figure 3b)

$$\begin{aligned} r_j^{(l)} &\sim \text{Bernoulli}(p), \\ \tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \\ z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)}, \\ y_i^{(l+1)} &= f(z_i^{(l+1)}). \end{aligned}$$

我们同时证明，添加噪声不仅对无监督特征学习有帮助，还能够被应用到有监督学习问题中。事实上，我们的方法还可以用于其他基于神经元的架构，例如玻尔兹曼机。尽管对于去噪自编码器来说5%的噪声强度通常最为理想，但通过我们在测试阶段采用的权重缩放流程，就可以使用更高强度的噪声。实践显示，丢弃20%的输入单元和50%的隐藏单元往往是最佳选择。

由于丢弃机制可被视为一种随机正则化技术，因此很自然地会考虑通过剔除噪声来得到的其确定性对应形式。在本文中，我们证明了在简单情况下，可以通过解析方式对丢弃机制进行边缘化处理，从而获得确定性正则化方法。近期，范德马滕等人（2013年）也研究了对应于不同指数族噪声分布的确定性正则化器，其中包括他们称之为“空白噪声”的丢弃机制。不过，他们的研究是将噪声施加到输入数据上，并且仅探讨了没有隐藏层的模型。Wang与Manning（2013年）则提出了一种通过边缘化丢弃噪声来加速该机制运行的方法。而Chen等人（2012年）则在去噪自编码器的背景下研究了类似的边缘化处理方法。

在丢弃机制中，我们是在噪声分布的约束下以随机方式最小化损失函数，这实际上等价于最小化期望损失函数。此前，格洛伯森与罗伊斯（2006年）以及德克尔等人（2010年）探讨过另一种设定：此时损失函数会在对手决定要丢弃哪些单元的情况下被最小化。在这种设定中，并不存在噪声分布，而是固定了最多可以丢弃的单元数量。然而，这些研究同样没有涉及包含隐藏单元的模型。

4. 模型描述

本节将对采用丢弃机制的神经网络模型进行说明。假设该神经网络包含 L 个隐藏层，用 $l \in \{1, \dots, L\}$ 来标记这些隐藏层的序号。设 $\mathbf{z}^{(l)}$ 表示输入到第 l 层的向量， $\mathbf{y}^{(l)}$ 表示第 l 层的输出向量（其中 $\mathbf{y}^{(0)} = \mathbf{x}$ 即为输入）。 $W^{(l)}$ 和 $\mathbf{b}^{(l)}$ 分别代表第 l 层的权重与偏置。标准神经网络的前向传播操作（见图3a）可表述为：对于任意隐藏单元 i 以及 $l \in \{0, \dots, L-1\}$,

$$\begin{aligned} z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \mathbf{y}^l + b_i^{(l+1)}, \\ y_i^{(l+1)} &= f(z_i^{(l+1)}), \end{aligned}$$

其中 f 为任意激活函数，例如 $f(x) = 1/(1 + \exp(-x))$ 。

在采用丢弃机制后，前向传播操作会变为如图3b所示。

$$\begin{aligned} r_j^{(l)} &\sim \text{Bernoulli}(p), \\ \tilde{\mathbf{y}}^{(l)} &= \mathbf{r}^{(l)} * \mathbf{y}^{(l)}, \\ z_i^{(l+1)} &= \mathbf{w}_i^{(l+1)} \tilde{\mathbf{y}}^l + b_i^{(l+1)}, \\ y_i^{(l+1)} &= f(z_i^{(l+1)}). \end{aligned}$$

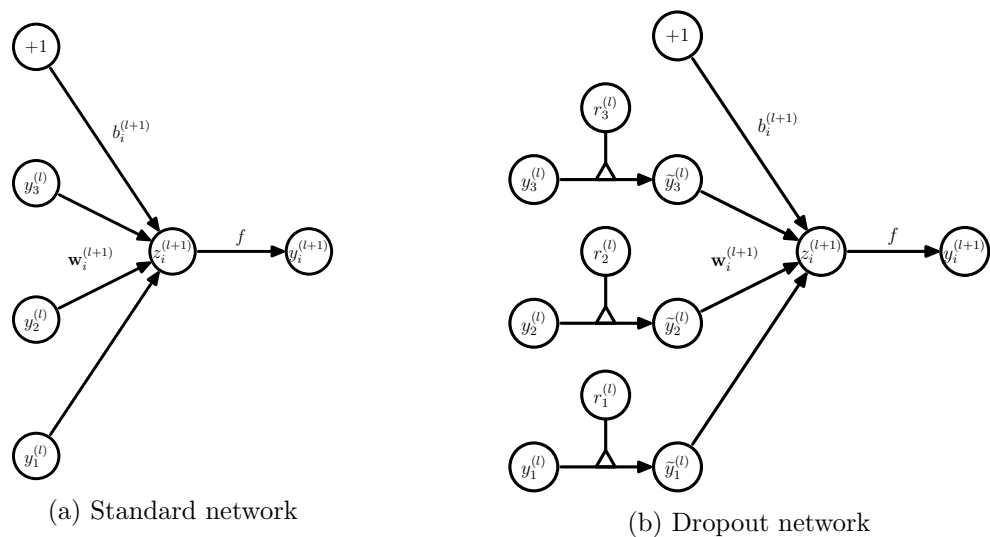


Figure 3: Comparison of the basic operations of a standard and dropout network.

Here $*$ denotes an element-wise product. For any layer l , $\mathbf{r}^{(l)}$ is a vector of independent Bernoulli random variables each of which has probability p of being 1. This vector is sampled and multiplied element-wise with the outputs of that layer, $\mathbf{y}^{(l)}$, to create the thinned outputs $\tilde{\mathbf{y}}^{(l)}$. The thinned outputs are then used as input to the next layer. This process is applied at each layer. This amounts to sampling a sub-network from a larger network. For learning, the derivatives of the loss function are backpropagated through the sub-network. At test time, the weights are scaled as $W_{test}^{(l)} = pW^{(l)}$ as shown in Figure 2. The resulting neural network is used without dropout.

5. Learning Dropout Nets

This section describes a procedure for training dropout neural nets.

5.1 Backpropagation

Dropout neural networks can be trained using stochastic gradient descent in a manner similar to standard neural nets. The only difference is that for each training case in a mini-batch, we sample a thinned network by dropping out units. Forward and backpropagation for that training case are done only on this thinned network. The gradients for each parameter are averaged over the training cases in each mini-batch. Any training case which does not use a parameter contributes a gradient of zero for that parameter. Many methods have been used to improve stochastic gradient descent such as momentum, annealed learning rates and L2 weight decay. Those were found to be useful for dropout neural networks as well.

One particular form of regularization was found to be especially useful for dropout—constraining the norm of the incoming weight vector at each hidden unit to be upper bounded by a fixed constant c . In other words, if $\mathbf{w}$ represents the vector of weights incident on any hidden unit, the neural network was optimized under the constraint $\|\mathbf{w}\|_2 \leq c$. This constraint was imposed during optimization by projecting $\mathbf{w}$ onto the surface of a ball of radius c , whenever $\mathbf{w}$ went out of it. This is also called max-norm regularization since it implies that the maximum value that the norm of any weight can take is c . The constant

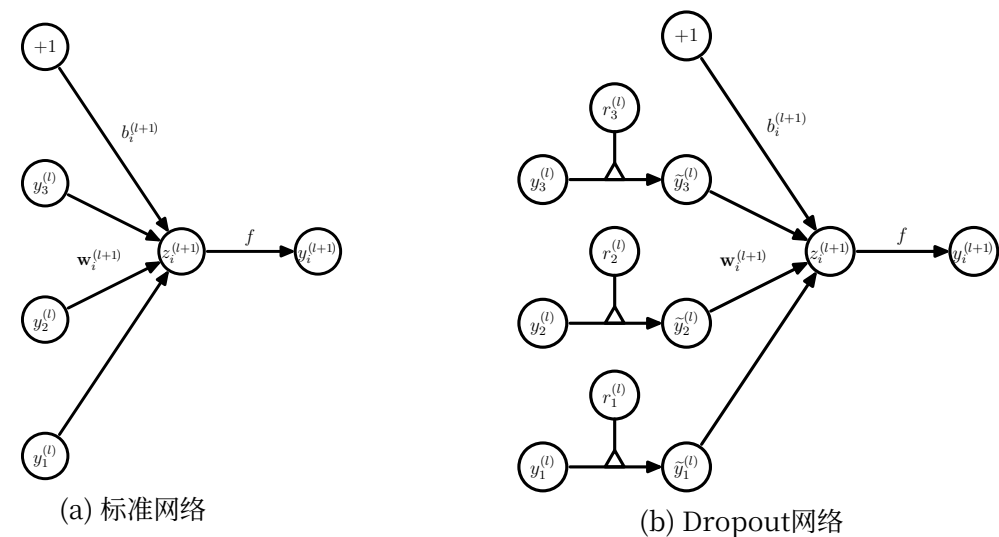


图3: 标准网络与Dropout网络的基本运算对比。

此处 $*$ 表示元素级乘积。对于任意一层 l , $\mathbf{r}^{(l)}$ 是一个由独立的伯努利随机变量构成的向量，每个变量为1的概率为 p 。该向量会被抽样出来，并与该层的输出 $\mathbf{y}^{(l)}$ 进行元素级乘法运算，从而生成经过缩减的输出 $\tilde{\mathbf{y}}^{(l)}$ 。这些缩减后的输出随后会被作为下一层的输入。这一过程会在每一层都重复执行，实际上等同于从更大的网络中抽取出一个子网络。在训练过程中，损失函数的梯度会通过这个子网络进行反向传播；而在测试阶段，则会按照图2所示的方式对权重进行缩放，即 $W^{(l)}_{test} = pW^{(l)}$ 。最终得到的神经网络是不使用丢弃机制的版本。

5. 学习Dropout神经网络

本节介绍了训练带有丢弃机制的神经网络的流程。

5.1 反向传播

丢弃参数可以通过随机梯度下降的方式进行训练，其方式与标准神经网络类似。唯一的区别在于，在每个小批量的训练案例中，我们会通过丢弃某些单元来构建一个简化网络，而针对该训练案例的前向传播和反向传播仅在这个简化网络上进行。随后会将每个参数在所有小批量训练案例中的梯度取平均值；那些未使用该参数的训练案例则会为该参数贡献零梯度。人们已经采用了多种方法来改进随机梯度下降，比如动量、退火学习率以及L2权重衰减，而这些方法也被证明对丢弃神经网络同样有效。

有一种特定形式的正则化方法被证明对丢弃机制尤为有效——它通过将每个隐藏单元所对应的权重向量的最大范数限制在某个固定常数 c 以内来实现约束。换言之，如果 $\mathbf{w}$ 代表作用在任意隐藏单元上的权重向量，那么该神经网络就会在 $\|\mathbf{w}\|_2 \leq c$ 这一约束条件下进行优化。在优化过程中，一旦 $\mathbf{w}$ 的值超出某个半径为 c 的球面范围，就会通过将其投影到该球面上来施加这一约束。由于这种做法要求任何权重向量的范数最大值都不能超过 c ，因此它也被称作最大范数正则化。而这个常数

c is a tunable hyperparameter, which is determined using a validation set. Max-norm regularization has been previously used in the context of collaborative filtering (Srebro and Shraibman, 2005). It typically improves the performance of stochastic gradient descent training of deep neural nets, even when no dropout is used.

Although dropout alone gives significant improvements, using dropout along with max-norm regularization, large decaying learning rates and high momentum provides a significant boost over just using dropout. A possible justification is that constraining weight vectors to lie inside a ball of fixed radius makes it possible to use a huge learning rate without the possibility of weights blowing up. The noise provided by dropout then allows the optimization process to explore different regions of the weight space that would have otherwise been difficult to reach. As the learning rate decays, the optimization takes shorter steps, thereby doing less exploration and eventually settles into a minimum.

5.2 Unsupervised Pretraining

Neural networks can be pretrained using stacks of RBMs (Hinton and Salakhutdinov, 2006), autoencoders (Vincent et al., 2010) or Deep Boltzmann Machines (Salakhutdinov and Hinton, 2009). Pretraining is an effective way of making use of unlabeled data. Pretraining followed by finetuning with backpropagation has been shown to give significant performance boosts over finetuning from random initializations in certain cases.

Dropout can be applied to finetune nets that have been pretrained using these techniques. The pretraining procedure stays the same. The weights obtained from pretraining should be scaled up by a factor of $1/p$. This makes sure that for each unit, the expected output from it under random dropout will be the same as the output during pretraining. We were initially concerned that the stochastic nature of dropout might wipe out the information in the pretrained weights. This did happen when the learning rates used during finetuning were comparable to the best learning rates for randomly initialized nets. However, when the learning rates were chosen to be smaller, the information in the pretrained weights seemed to be retained and we were able to get improvements in terms of the final generalization error compared to not using dropout when finetuning.

6. Experimental Results

We trained dropout neural networks for classification problems on data sets in different domains. We found that dropout improved generalization performance on *all* data sets compared to neural networks that did not use dropout. Table 1 gives a brief description of the data sets. The data sets are

- MNIST : A standard toy data set of handwritten digits.
- TIMIT : A standard speech benchmark for clean speech recognition.
- CIFAR-10 and CIFAR-100 : Tiny natural images (Krizhevsky, 2009).
- Street View House Numbers data set (SVHN) : Images of house numbers collected by Google Street View (Netzer et al., 2011).
- ImageNet : A large collection of natural images.
- Reuters-RCV1 : A collection of Reuters newswire articles.

c 是一个可通过验证集来确定值的可调超参数。最大范数正则化此前已在协同过滤领域被应用过 (Srebro and Shraibman, 2005年)。即便不使用丢弃机制, 它通常也能提升深度神经网络采用随机梯度下降法训练时的性能。

尽管仅使用丢弃机制就能带来显著提升, 但若将丢弃机制与最大范数正则化、逐渐衰减的学习率以及较高的动量相结合, 其效果会比单独使用丢弃机制更为出色。这一优势的潜在原因在于, 通过将权重向量限制在固定半径的球体范围内, 即便采用极高的学习率, 权重也不会出现爆炸性增长的情况。与此同时, 丢弃机制产生的噪声能够帮助优化过程探索权重空间中那些原本难以触及的区域。随着学习率逐渐降低, 优化过程的步长也会随之缩小, 探索范围相应减少, 最终收敛到最小值处。

5.2 无监督预训练

神经网络可以通过堆叠受限玻尔兹曼机 (Hinton和Salakhutdinov, 2006)、自编码器 (Vincent等人, 2010) 或深度玻尔兹曼机 (Salakhutdinov和Hinton, 2009) 来进行预训练。预训练是一种有效利用未标记数据的方法。在某些情况下, 先进行预训练再通过反向传播进行微调, 所带来的性能提升显著优于直接从随机初始状态开始微调。

可以将丢弃机制用于对经过这些预训练技术处理过的神经网络进行微调。预训练的流程保持不变, 只需将预训练所得到的权重乘以 $1/p$ 这个系数即可。这样做能够确保在随机应用丢弃机制的情况下, 每个单元的预期输出与预训练时的输出保持一致。起初我们担心丢弃机制的随机特性会抹去预训练权重中存储的信息, 而当微调时所使用的学习率与随机初始化神经网络的最佳学习率相近时, 这种情况确实会出现。不过, 若选择较小的学习率, 预训练权重中的信息似乎仍能得以保留, 相比在微调时不使用丢弃机制, 最终模型的泛化误差也能得到改善。

6. 实验结果

我们在不同领域的数据集上, 针对分类任务训练了采用丢弃机制的丢弃神经网络。实验结果表明, 与未使用丢弃机制的神经网络相比, 丢弃机制提升了所有数据集上的泛化性能。表1对这些数据集进行了简要说明, 具体数据集内容如下。

- MNIST: 一个用于手写数字训练的标准示例数据集。
- TIMIT: 用于高质量语音识别测试的标准语音基准数据集。
- CIFAR-10与CIFAR-100: 由克里兹夫斯基在2009年提出的小型自然图像数据集。
- 街景房屋编号数据集 (SVHN): 由谷歌街景采集的房屋编号图像, 相关研究由Netzer等人在2011年发表。
- ImageNet: 一个包含海量自然图像的大型数据集。
- Reuters-RCV1: 一组来自路透社的新闻文章。

- Alternative Splicing data set: RNA features for predicting alternative gene splicing (Xiong et al., 2011).

We chose a diverse set of data sets to demonstrate that dropout is a general technique for improving neural nets and is not specific to any particular application domain. In this section, we present some key results that show the effectiveness of dropout. A more detailed description of all the experiments and data sets is provided in Appendix B.

Data Set	Domain	Dimensionality	Training Set	Test Set
MNIST	Vision	784 (28×28 grayscale)	60K	10K
SVHN	Vision	3072 (32×32 color)	600K	26K
CIFAR-10/100	Vision	3072 (32×32 color)	60K	10K
ImageNet (ILSVRC-2012)	Vision	65536 (256×256 color)	1.2M	150K
TIMIT	Speech	2520 (120-dim, 21 frames)	1.1M frames	58K frames
Reuters-RCV1	Text	2000	200K	200K
Alternative Splicing	Genetics	1014	2932	733

Table 1: Overview of the data sets used in this paper.

6.1 Results on Image Data Sets

We used five image data sets to evaluate dropout—MNIST, SVHN, CIFAR-10, CIFAR-100 and ImageNet. These data sets include different image types and training set sizes. Models which achieve state-of-the-art results on *all* of these data sets use dropout.

6.1.1 MNIST

Method	Unit Type	Architecture	Error %
Standard Neural Net (Simard et al., 2003)	Logistic	2 layers, 800 units	1.60
SVM Gaussian kernel	NA	NA	1.40
Dropout NN	Logistic	3 layers, 1024 units	1.35
Dropout NN	ReLU	3 layers, 1024 units	1.25
Dropout NN + max-norm constraint	ReLU	3 layers, 1024 units	1.06
Dropout NN + max-norm constraint	ReLU	3 layers, 2048 units	1.04
Dropout NN + max-norm constraint	ReLU	2 layers, 4096 units	1.01
Dropout NN + max-norm constraint	ReLU	2 layers, 8192 units	0.95
Dropout NN + max-norm constraint (Goodfellow et al., 2013)	Maxout	2 layers, (5×240) units	0.94
DBN + finetuning (Hinton and Salakhutdinov, 2006)	Logistic	500-500-2000	1.18
DBM + finetuning (Salakhutdinov and Hinton, 2009)	Logistic	500-500-2000	0.96
DBN + dropout finetuning	Logistic	500-500-2000	0.92
DBM + dropout finetuning	Logistic	500-500-2000	0.79

Table 2: Comparison of different models on MNIST.

The MNIST data set consists of 28×28 pixel handwritten digit images. The task is to classify the images into 10 digit classes. Table 2 compares the performance of dropout with other techniques. The best performing neural networks for the permutation invariant

- 可变剪接数据集：用于预测基因可变剪接的RNA特征（熊等人，2011年）。

我们选取了多种不同的数据集，以此证明丢弃机制是一种通用的技术，可用于提升各种神经网络的性能，而并非仅适用于某一特定的应用领域。在本节中，我们将展示一些关键结果，以印证丢弃机制的有效性。所有实验及数据集的更详细说明可见附录B。

数据集	领域	维度	训练集	测试集
MNIST数据集	视觉任务	784 (28×28 灰度图)	60K	10K
SVHN	视觉任务	3072 (32×32 彩色图)	600K	26K
CIFAR-10/100数据集	视觉任务	3072 (32×32 种颜色)	60K	10K
ImageNet数据集 (ILSVRC-2012竞赛)	视觉任务	65536 (256×256 种颜色)	1.2M	150K
TIMIT数据集	语音数据	2520样本数据集 (120维, 共21帧)	110万帧	5.8万帧
Reuters-RCV1数据集	Text	2000	200K	200K
可变剪接	遗传学	1014	2932	733

表1: 本文所使用的数据集概览。

6.1 图像数据集上的实验结果

我们采用了五个图像数据集来评估丢弃机制的效果——即MNIST、SVHN、CIFAR-10、CIFAR-100以及ImageNet数据集。这些数据集包含不同类型的图像以及不同的训练集规模。那些在所有这些数据集上都能取得最先进性能的模型，均采用了丢弃机制。

6.1.1 MNIST

方法	Unit Type	网络架构	误差 %
标准神经网络 (Simard等人, 2003年)	逻辑回归	共2层, 包含800个单元	1.60
支持向量机高斯核	NA	NA	1.40
dropout神经网络	逻辑回归	3层结构, 共1024个单元	1.35
dropout神经网络	ReLU	3层结构, 共1024个单元	1.25
dropout神经网络 + 最大范数约束	ReLU	3层结构, 包含1024个单元	1.06
dropout神经网络采用 + 最大范数约束机制	ReLU	3层结构, 包含2048个单元	1.04
dropout神经网络采用 + 最大范数约束机制	ReLU	2层结构, 包含4096个单元	1.01
dropout神经网络采用 + 最大范数约束机制	ReLU	共2层, 包含8192个单元。	0.95
dropout神经网络采用最大范数约束, 这一方法由Goodfellow等人于2013年提出。(et al., 2013年)	Maxout激活函数	同样为2层, 其 5×240 值为。 单元	0.94
DBN的 + 微调方法 (欣顿与萨拉胡特丁诺夫, 2006)	逻辑回归	500-500-2000	1.18
DBM模型的 + 微调方法 (萨拉胡特丁诺夫与欣顿, 2009)	逻辑回归	500-500-2000	0.96
DBN + 结合丢弃机制的微调	逻辑回归	500-500-2000	0.92
DBN + 结合丢弃机制的微调	逻辑回归	500-500-2000	0.79

表2: 不同模型在MNIST数据集上的性能对比。

MNIST数据集由 28×28 像素大小的手写数字图像组成。该任务的目标是将这些图像分类到10个不同的数字类别中。表2对比了丢弃机制与其他技术的相关性能表现。在处理排列不变性问题上，性能最佳的神经网络为

setting that do not use dropout or unsupervised pretraining achieve an error of about 1.60% (Simard et al., 2003). With dropout the error reduces to 1.35%. Replacing logistic units with rectified linear units (ReLUs) (Jarrett et al., 2009) further reduces the error to 1.25%. Adding max-norm regularization again reduces it to 1.06%. Increasing the size of the network leads to better results. A neural net with 2 layers and 8192 units per layer gets down to 0.95% error. Note that this network has more than 65 million parameters and is being trained on a data set of size 60,000. Training a network of this size to give good generalization error is very hard with standard regularization methods and early stopping. Dropout, on the other hand, prevents overfitting, even in this case. It does not even need early stopping. Goodfellow et al. (2013) showed that results can be further improved to 0.94% by replacing ReLU units with maxout units. All dropout nets use $p = 0.5$ for hidden units and $p = 0.8$ for input units. More experimental details can be found in Appendix B.1.

Dropout nets pretrained with stacks of RBMs and Deep Boltzmann Machines also give improvements as shown in Table 2. DBM—pretrained dropout nets achieve a test error of 0.79% which is the best performance ever reported for the permutation invariant setting. We note that it is possible to obtain better results by using 2-D spatial information and augmenting the training set with distorted versions of images from the standard training set. We demonstrate the effectiveness of dropout in that setting on more interesting data sets.

In order to test the robustness of dropout, classification experiments were done with networks of many different architectures keeping all hyperparameters, including p , fixed. Figure 4 shows the test error rates obtained for these different architectures as training progresses. The same architectures trained with and without dropout have drastically different test errors as seen as by the two separate clusters of trajectories. Dropout gives a huge improvement across all architectures, without using hyperparameters that were tuned specifically for each architecture.

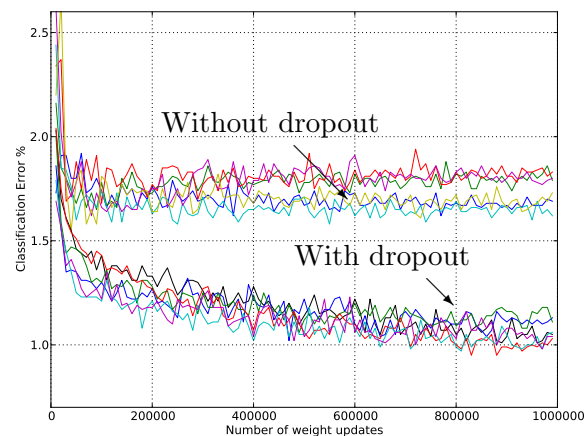


Figure 4: Test error for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

6.1.2 STREET VIEW HOUSE NUMBERS

The Street View House Numbers (SVHN) Data Set (Netzer et al., 2011) consists of color images of house numbers collected by Google Street View. Figure 5a shows some examples of images from this data set. The part of the data set that we use in our experiments consists of 32×32 color images roughly centered on a digit in a house number. The task is to identify that digit.

For this data set, we applied dropout to convolutional neural networks (LeCun et al., 1989). The best architecture that we found has three convolutional layers followed by 2 fully connected hidden layers. All hidden units were ReLUs. Each convolutional layer was

所有Dropout神经网络均采用 $p = 0.5$ 作为隐藏单元的激活函数, $p = 0.8$ 作为输入单元的激活函数。有关更多的实验细节, 可以参考附录B.1。Goodfellow等人 (2013年) 证明, 通过将ReLU单元替换为Maxout单元, 模型的性能还能进一步提升, 误差可降至0.94%。

如表2所示, 采用由受限玻尔兹曼机与深度玻尔兹曼机构成的堆叠结构进行预训练的丢弃机制的神经网络同样能够带来性能提升。经过预训练的DBM模型对应的测试误差仅为0.79%, 这一数值是迄今为止在排列不变设置下所记录的最佳性能。需要指出的是, 通过利用二维空间信息, 并用标准训练集中图像的失真版本来扩充训练集, 还有可能获得更优的结果。我们将在更多有趣的数据集上进一步验证该设置下丢弃机制的有效性。

为检验丢弃机制的鲁棒性, 我们使用了多种不同架构的网络开展了分类实验, 且所有超参数, 包括 p , 都保持不变。图4展示了随着训练进程推进, 这些不同架构所对应的测试误差变化情况。从两组独立的误差曲线可以看出, 采用丢弃机制与不采用该机制训练出的相同架构, 其测试误差存在巨大差异。即便没有针对每种架构单独调整超参数, 丢弃机制依然能在所有架构上带来显著的性能提升。

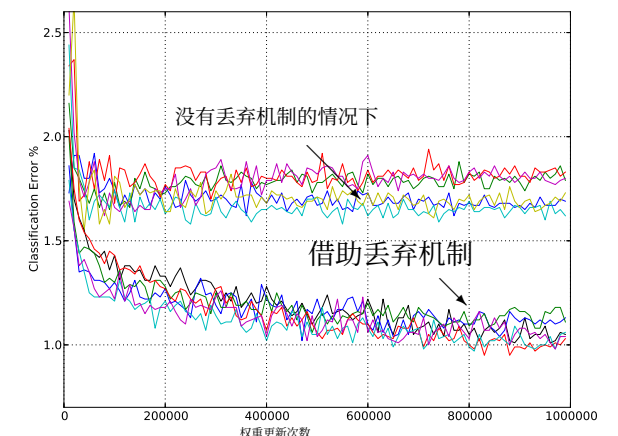


图4: 不同网络架构在启用及未启用丢弃机制时的测试误差情况。这些网络均拥有2到4层隐藏层, 每层的单元数量则在1024到2048个之间。

6.1.2 STREET VIEW HOUSE NUMBERS

SVHN数据集 (Netzer等人, 2011年) 由通过谷歌街景收集的房屋编号彩色图像组成。

图5a展示了该数据集中的一些图像示例。我们在实验中使用的部分数据集包含 32×32 张彩色图像, 这些图像的焦点大致位于房屋编号中的某个数字上, 任务就是识别出这个数字。

针对该数据集, 我们在卷积神经网络 (勒昆等人, 1989年) 中应用了丢弃机制。我们找到的最佳网络架构包含三层卷积层, 其后接两层全连接隐藏层, 所有隐藏层均采用修正线性单元作为激活函数, 且每层卷积层后还设有最大池化层。附录B.2对该网络架构有更详细的描述。

Method	Error %
Binary Features (WDCH) (Netzer et al., 2011)	36.7
HOG (Netzer et al., 2011)	15.0
Stacked Sparse Autoencoders (Netzer et al., 2011)	10.3
KMeans (Netzer et al., 2011)	9.4
Multi-stage Conv Net with average pooling (Sermanet et al., 2012)	9.06
Multi-stage Conv Net + L2 pooling (Sermanet et al., 2012)	5.36
Multi-stage Conv Net + L4 pooling + padding (Sermanet et al., 2012)	4.90
Conv Net + max-pooling	3.95
Conv Net + max pooling + dropout in fully connected layers	3.02
Conv Net + stochastic pooling (Zeiler and Fergus, 2013)	2.80
Conv Net + max pooling + dropout in all layers	2.55
Conv Net + maxout (Goodfellow et al., 2013)	2.47
Human Performance	2.0

Table 3: Results on the Street View House Numbers data set.

followed by a max-pooling layer. Appendix B.2 describes the architecture in more detail. Dropout was applied to all the layers of the network with the probability of retaining a hidden unit being $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ for the different layers of the network (going from input to convolutional layers to fully connected layers). Max-norm regularization was used for weights in both convolutional and fully connected layers. Table 3 compares the results obtained by different methods. We find that convolutional nets outperform other methods. The best performing convolutional nets that do not use dropout achieve an error rate of 3.95%. Adding dropout only to the fully connected layers reduces the error to 3.02%. Adding dropout to the convolutional layers as well further reduces the error to 2.55%. Even more gains can be obtained by using maxout units.

The additional gain in performance obtained by adding dropout in the convolutional layers (3.02% to 2.55%) is worth noting. One may have presumed that since the convolutional layers don’t have a lot of parameters, overfitting is not a problem and therefore dropout would not have much effect. However, dropout in the lower layers still helps because it provides noisy inputs for the higher fully connected layers which prevents them from overfitting.

6.1.3 CIFAR-10 AND CIFAR-100

The CIFAR-10 and CIFAR-100 data sets consist of 32×32 color images drawn from 10 and 100 categories respectively. Figure 5b shows some examples of images from this data set. A detailed description of the data sets, input preprocessing, network architectures and other experimental details is given in Appendix B.3. Table 4 shows the error rate obtained by different methods on these data sets. Without any data augmentation, Snoek et al. (2012) used Bayesian hyperparameter optimization to obtained an error rate of 14.98% on CIFAR-10. Using dropout in the fully connected layers reduces that to 14.32% and adding dropout in every layer further reduces the error to 12.61%. Goodfellow et al. (2013) showed that the error is further reduced to 11.68% by replacing ReLU units with maxout units. On CIFAR-100, dropout reduces the error from 43.48% to 37.20% which is a huge improvement. No data augmentation was used for either data set (apart from the input dropout).

方法	误差百分比
二进制特征（WDCH）（Netzer等人，2011年）	36.7
HOG特征（Netzer等人，2011年）	15.0
堆叠稀疏自编码器（Netzer等人，2011年）	10.3
K均值算法（Netzer等人，2011年）	9.4
带平均池化的多阶段卷积网络（Sermanet等人，2012年）	9.06
带L2正则化的多阶段卷积网络（Sermanet等人，2012年）	5.36
采用L4正则化及填充处理的多阶段卷积网络（Sermanet等人，2012年）	4.90
卷积神经网络配合最大池化技术	3.95
卷积神经网络结合最大池化以及全连接层中的丢弃机制	3.02
卷积神经网络 + 结合随机池化技术（扎伊勒与弗格斯，2013年）	2.80
卷积神经网络 +采用最大池化以及所有层中的丢弃机制	2.55
卷积神经网络 +使用Maxout激活函数（Goodfellow等人，2013年）	2.47
人类性能指标	2.0

表3：街景房屋编号数据集上的实验结果。

对于网络从输入层到卷积层再到全连接层的不同层次，保留每个隐藏单元的概率分别为 $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ ，并且卷积层与全连接层的权重都采用了最大范数正则化处理。表3对比了不同方法所得的结果，结果显示卷积网络的性能优于其他方法。那些未使用丢弃机制但性能最佳的卷积网络的错误率为3.95%，仅在全连接层应用丢弃机制时错误率可降至3.02%，若同时在卷积层也应用丢弃机制，则错误率还能进一步降低至2.55%。而使用Maxout激活函数的单元则能带来更高的性能提升。

值得注意的是，在卷积层中加入丢弃机制后，性能还能进一步提升，提升幅度在3.02%到2.55%之间。人们或许会认为，由于卷积层的参数数量并不多，过拟合问题并不突出，因此丢弃机制的作用应该有限。但实际上，即使在较低层次的卷积层中应用丢弃机制也能发挥作用，因为它能为更高层次的全连接层提供带有噪声的输入，从而有效防止这些层出现过拟合现象。

6.1.3 CIFAR-10 AND CIFAR-100

CIFAR-10数据集和CIFAR-100数据集分别包含 32×32 张来自10类和100类的彩色图像。图5b展示了该数据集的一些图像示例。关于这些数据集的详细描述、输入预处理方式、网络架构以及其他实验细节，均记载于附录B.3中。表4列出了不同方法在这些数据集上所得的错误率。Snoek等人（2012年）在未采用任何数据增强措施的情况下，通过贝叶斯超参数优化方法，在CIFAR-10数据集上将错误率降至14.98%。在全连接层中引入丢弃机制后，这一错误率降至14.32%，而如果在每一层都加入丢弃机制，错误率还能进一步下降到12.61%。Goodfellow等人（2013年）的研究表明，用Maxout激活函数替换ReLU激活函数单元后，错误率可进一步降至11.68%。在CIFAR-100数据集上，丢弃机制使得错误率从43.48%降低到了37.20%，提升了相当大的幅度。值得注意的是，除了在输入层应用丢弃机制外，这两组数据集均未进行任何数据增强处理。



(a) Street View House Numbers (SVHN) (b) CIFAR-10

Figure 5: Samples from image data sets. Each row corresponds to a different category.

Method	CIFAR-10	CIFAR-100
Conv Net + max pooling (hand tuned)	15.60	43.48
Conv Net + stochastic pooling (Zeiler and Fergus, 2013)	15.13	42.51
Conv Net + max pooling (Snoek et al., 2012)	14.98	-
Conv Net + max pooling + dropout fully connected layers	14.32	41.26
Conv Net + max pooling + dropout in all layers	12.61	37.20
Conv Net + maxout (Goodfellow et al., 2013)	11.68	38.57

Table 4: Error rates on CIFAR-10 and CIFAR-100.

6.1.4 IMAGENET

ImageNet is a data set of over 15 million labeled high-resolution images belonging to roughly 22,000 categories. Starting in 2010, as part of the Pascal Visual Object Challenge, an annual competition called the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) has been held. A subset of ImageNet with roughly 1000 images in each of 1000 categories is used in this challenge. Since the number of categories is rather large, it is conventional to report two error rates: top-1 and top-5, where the top-5 error rate is the fraction of test images for which the correct label is not among the five labels considered most probable by the model. Figure 6 shows some predictions made by our model on a few test images.

ILSVRC-2010 is the only version of ILSVRC for which the test set labels are available, so most of our experiments were performed on this data set. Table 5 compares the performance of different methods. Convolutional nets with dropout outperform other methods by a large margin. The architecture and implementation details are described in detail in Krizhevsky et al. (2012).



(a) 街景房屋编号 (SVHN) (b) CIFAR-10

图5：来自各数据集的样本。每行代表一个不同的类别。

方法	CIFAR-10	CIFAR-100数据集
卷积神经网络 + 搭配最大池化（手动调整参数）	15.60	43.48
卷积神经网络 + 搭配随机池化（出自Zeiler与Fergus在2013年的研究）	15.13	42.51
卷积神经网络 + 搭配最大池化（由Snoek等人于2012年提出）	14.98	-
卷积神经网络 + 搭配最大池化、 + 丢弃机制以及全连接层	14.32	41.26
卷积神经网络 + 搭配最大池化、 + 丢弃机制，并应用于所有层	12.61	37.20
卷积神经网络 +搭配Maxout激活函数（出自Goodfellow等人2013年的研究）	11.68	38.57

表4：CIFAR-10数据集与CIFAR-100数据集上的错误率。

6.1.4 IMAGENET

ImageNet数据集拥有超过1500万张经过标注的高分辨率图像，这些图像大致属于22,000个不同的类别。自2010年起，作为Pascal视觉物体识别挑战赛的一部分，一项名为ImageNet大规模视觉识别挑战赛（ILSVRC）的年度竞赛便开始举办。该竞赛使用的数据集是从ImageNet中抽取的子集，其中每个类别约有1000张图像。由于类别数量极为庞大，业界通常会报告两种错误率：top-1错误率和top-5错误率，其中top-5错误率指的是测试图像中正确标签不在模型认为最可能的五个标签之中的比例。图6展示了我们的模型对几张测试图像所做的预测结果。

由于只有ILSVRC-2010年的测试集标签是公开的，因此我们的大部分实验都是在该数据集上进行的。表5对比了不同方法的性能，结果表明采用丢弃机制的卷积网络远远优于其他方法。其网络架构及实现细节在Krizhevsky等人（2012年）的研究中有详细阐述。



Figure 6: Some ImageNet test cases with the 4 most probable labels as predicted by our model. The length of the horizontal bars is proportional to the probability assigned to the labels by the model. Pink indicates ground truth.

Model	Top-1	Top-5
Sparse Coding (Lin et al., 2010)	47.1	28.2
SIFT + Fisher Vectors (Sanchez and Perronnin, 2011)	45.7	25.7
Conv Net + dropout (Krizhevsky et al., 2012)	37.5	17.0

Table 5: Results on the ILSVRC-2010 test set.

Model	Top-1 (val)	Top-5 (val)	Top-5 (test)
SVM on Fisher Vectors of Dense SIFT and Color Statistics	-	-	27.3
Avg of classifiers over FVs of SIFT, LBP, GIST and CSIFT	-	-	26.2
Conv Net + dropout (Krizhevsky et al., 2012)	40.7	18.2	-
Avg of 5 Conv Nets + dropout (Krizhevsky et al., 2012)	38.1	16.4	16.4

Table 6: Results on the ILSVRC-2012 validation/test set.

Our model based on convolutional nets and dropout won the ILSVRC-2012 competition. Since the labels for the test set are not available, we report our results on the test set for the final submission and include the validation set results for different variations of our model. Table 6 shows the results from the competition. While the best methods based on standard vision features achieve a top-5 error rate of about 26%, convolutional nets with dropout achieve a test error of about 16% which is a staggering difference. Figure 6 shows some examples of predictions made by our model. We can see that the model makes very reasonable predictions, even when its best guess is not correct.

6.2 Results on TIMIT

Next, we applied dropout to a speech recognition task. We use the TIMIT data set which consists of recordings from 680 speakers covering 8 major dialects of American English reading ten phonetically-rich sentences in a controlled noise-free environment. Dropout neural networks were trained on windows of 21 log-filter bank frames to predict the label of the central frame. No speaker dependent operations were performed. Appendix B.4 describes the data preprocessing and training details. Table 7 compares dropout neural



图6: 展示了由我们的模型预测出的4个最可能的标签对应的部分ImageNet测试案例。水平条的长度与模型为这些标签赋予的概率成正比，粉色则表示真实标签。

模型	Top-1	Top-5
稀疏编码算法（林等人，2010年）	47.1	28.2
SIFT + 费希尔向量（桑切斯与佩罗宁，2011年）	45.7	25.7
卷积神经网络 + 丢弃机制（克里兹夫斯基等人，2012年）	37.5	17.0

表5: ILSVRC-2010测试集上的实验结果。

模型	top-1 (验证集)	top-5错误率 (验证集)	top-5错误率 (测试集)
基于密集版SIFT算法的费希尔向量与颜色统计量的支持向量机模型	-	-	27.3
基于SIFT、LBP、灰度图像结构特征及彩色SIFT算法的费希尔向量的分类器平均值	-	-	26.2
卷积神经网络 + 丢弃机制（克里兹夫斯基等人，2012年）	40.7	18.2	-
5个应用了 + 丢弃机制的卷积神经网络的平均值（克里兹夫斯基等人，2012年）	38.1	16.4	16.4

表6: ILSVRC-2012竞赛测试集/验证集上的实验结果。

我们基于卷积网络与丢弃机制构建的模型赢得了ILSVRC-2012竞赛的冠军。鉴于该竞赛的测试集标签并未公开，我们在最终提交时仅汇报了在测试集上的测试结果，同时附上了模型不同版本在验证集上的表现数据。表6展示了竞赛的最终结果——虽然基于传统视觉特征的最佳方法的前五错误率约为26%，但采用丢弃机制的卷积网络的测试误差仅约为16%，两者差距极为显著。图6则展示了我们模型的一些预测示例，可见即便模型最准确的猜测出错，它给出的预测结果依然相当合理。

6.2 TIMIT数据集上的实验结果

接下来，我们在一项语音识别任务中应用了丢弃机制。该任务使用了TIMIT数据集，其中包含了680位说话者的录音，这些录音涵盖了美式英语的8种主要方言，所有内容都是在无噪声的受控环境中朗读的10个语音特征丰富的句子。我们利用带丢弃机制的神经网络，以21个对数滤波器组帧为窗口来训练模型，从而预测中心帧的标签，并且没有进行任何与特定说话者相关的处理。附录B.4详细介绍了数据预处理及训练的相关细节，表7则对比了带丢弃机制的神经网络与其他模型的性能表现。

nets with other models. A 6-layer net gives a phone error rate of 23.4%. Dropout further improves it to 21.8%. We also trained dropout nets starting from pretrained weights. A 4-layer net pretrained with a stack of RBMs get a phone error rate of 22.7%. With dropout, this reduces to 19.7%. Similarly, for an 8-layer net the error reduces from 20.5% to 19.7%.

Method	Phone Error Rate%
NN (6 layers) (Mohamed et al., 2010)	23.4
Dropout NN (6 layers)	21.8
DBN-pretrained NN (4 layers)	22.7
DBN-pretrained NN (6 layers) (Mohamed et al., 2010)	22.4
DBN-pretrained NN (8 layers) (Mohamed et al., 2010)	20.7
mcRBM-DBN-pretrained NN (5 layers) (Dahl et al., 2010)	20.5
DBN-pretrained NN (4 layers) + dropout	19.7
DBN-pretrained NN (8 layers) + dropout	19.7

Table 7: Phone error rate on the TIMIT core test set.

6.3 Results on a Text Data Set

To test the usefulness of dropout in the text domain, we used dropout networks to train a document classifier. We used a subset of the Reuters-RCV1 data set which is a collection of over 800,000 newswire articles from Reuters. These articles cover a variety of topics. The task is to take a bag of words representation of a document and classify it into 50 disjoint topics. Appendix B.5 describes the setup in more detail. Our best neural net which did not use dropout obtained an error rate of 31.05%. Adding dropout reduced the error to 29.62%. We found that the improvement was much smaller compared to that for the vision and speech data sets.

6.4 Comparison with Bayesian Neural Networks

Dropout can be seen as a way of doing an equally-weighted averaging of exponentially many models with shared weights. On the other hand, Bayesian neural networks (Neal, 1996) are the proper way of doing model averaging over the space of neural network structures and parameters. In dropout, each model is weighted equally, whereas in a Bayesian neural network each model is weighted taking into account the prior and how well the model fits the data, which is the more correct approach. Bayesian neural nets are extremely useful for solving problems in domains where data is scarce such as medical diagnosis, genetics, drug discovery and other computational biology applications. However, Bayesian neural nets are slow to train and difficult to scale to very large network sizes. Besides, it is expensive to get predictions from many large nets at test time. On the other hand, dropout neural nets are much faster to train and use at test time. In this section, we report experiments that compare Bayesian neural nets with dropout neural nets on a small data set where Bayesian neural networks are known to perform well and obtain state-of-the-art results. The aim is to analyze how much does dropout lose compared to Bayesian neural nets.

The data set that we use (Xiong et al., 2011) comes from the domain of genetics. The task is to predict the occurrence of alternative splicing based on RNA features. Alternative splicing is a significant cause of cellular diversity in mammalian tissues. Predicting the

一个6层结构的神经网络在语音识别任务中的音素错误率为234%, 而引入丢弃机制后这一数值降至218%。此外, 我们还尝试从预训练权重出发来训练带丢弃机制的神经网络: 使用由受限玻尔兹曼机堆叠而成的结构预训练的4层网络, 其音素错误率为227%, 加入丢弃机制后这一数值进一步下降至197%。同样的情况也出现在8层结构的神经网络上, 其错误率从205%降低到了197%。

方法	电话错误率百分比
神经网络 (6层) (Mohamed等人, 2010年)	23.4
dropout神经网络 (6层)	21.8
经过深度信念网络预训练的神经网络 (4层)	22.7
经过深度信念网络预训练的神经网络 (6层) (Mohamed等人, 2010年)	22.4
经过DBN预训练的神经网络 (8层) (Mohamed等人, 2010年)	20.7
经过mcRBM-DBN预训练的神经网络 (5层) (达尔等人, 2010年)	20.5
经过DBN预训练的神经网络 (4层) + 丢弃机制	19.7
经过DBN预训练的神经网络 (8层) + 丢弃机制	19.7

表7: TIMIT核心测试集上的电话错误率。

6.3 文本数据集上的实验结果

为验证丢弃机制在文本处理领域的实用性, 我们使用带丢弃机制的神经网络来训练一个文档分类器。实验所用的数据集是Reuters-RCV1数据集的子集, 该数据集汇集了来自路透社的80多万篇新闻文章, 内容涵盖多种主题。任务是将文档的词袋表示转换为相应形式, 再将其分类到50个互不重叠的主题中。附录B.5对实验设置进行了更详细的说明。未采用丢弃机制的最佳神经网络的错误率为3105%, 而加入丢弃机制后错误率降至2962%。我们发现, 与视觉任务和语音数据集相比, 这种改进的效果要小得多。

6.4 与贝叶斯神经网络的比较

丢弃机制可以被视作一种对数量呈指数级增长的、具有相同权重的模型进行等权重平均的方法。另一方面, 贝叶斯神经网络 (Neal, 1996) 则是针对神经网络结构与参数空间实现模型平均法的正确途径。在丢弃机制中, 所有模型的权重都是相同的; 而在贝叶斯神经网络中, 每个模型的权重会根据先验概率以及模型对数据的拟合程度来决定, 这才是更为准确的方法。对于那些数据较为稀缺的领域, 比如医学诊断、遗传学、药物研发以及各类计算生物学应用, 贝叶斯神经网络极具实用价值。不过, 这类网络的训练速度较慢, 且难以扩展到规模极大的网络结构, 同时在测试阶段从多个大型网络获取预测结果的成本也相当高。相比之下, 带丢弃机制的神经网络在训练速度和测试效率上都有显著优势。在本节中, 我们将在一个已知贝叶斯神经网络表现优异并能取得当前最佳结果的小型数据集上, 对比这两种网络的表现, 以此分析丢弃机制相比贝叶斯神经网络会损失多少性能。

我们所使用的数据集 (熊等人, 2011年) 源自遗传学领域。该任务旨在根据RNA特征预测可变剪接的发生情况。在哺乳动物组织中, 可变剪接是导致细胞多样性的重要因素。预测这种

Method	Code Quality (bits)
Neural Network (early stopping) (Xiong et al., 2011)	440
Regression, PCA (Xiong et al., 2011)	463
SVM, PCA (Xiong et al., 2011)	487
Neural Network with dropout	567
Bayesian Neural Network (Xiong et al., 2011)	623

Table 8: Results on the Alternative Splicing Data Set.

occurrence of alternate splicing in certain tissues under different conditions is important for understanding many human diseases. Given the RNA features, the task is to predict the probability of three splicing related events that biologists care about. The evaluation metric is Code Quality which is a measure of the negative KL divergence between the target and the predicted probability distributions (higher is better). Appendix B.6 includes a detailed description of the data set and this performance metric.

Table 8 summarizes the performance of different models on this data set. Xiong et al. (2011) used Bayesian neural nets for this task. As expected, we found that Bayesian neural nets perform better than dropout. However, we see that dropout improves significantly upon the performance of standard neural nets and outperforms all other methods. The challenge in this data set is to prevent overfitting since the size of the training set is small. One way to prevent overfitting is to reduce the input dimensionality using PCA. Thereafter, standard techniques such as SVMs or logistic regression can be used. However, with dropout we were able to prevent overfitting without the need to do dimensionality reduction. The dropout nets are very large (1000s of hidden units) compared to a few tens of units in the Bayesian network. This shows that dropout has a strong regularizing effect.

6.5 Comparison with Standard Regularizers

Several regularization methods have been proposed for preventing overfitting in neural networks. These include L2 weight decay (more generally Tikhonov regularization (Tikhonov, 1943)), lasso (Tibshirani, 1996), KL-sparsity and max-norm regularization. Dropout can be seen as another way of regularizing neural networks. In this section we compare dropout with some of these regularization methods using the MNIST data set.

The same network architecture (784-1024-1024-2048-10) with ReLUs was trained using stochastic gradient descent with different regularizations. Table 9 shows the results. The values of different hyperparameters associated with each kind of regularization (decay constants, target sparsity, dropout rate, max-norm upper bound) were obtained using a validation set. We found that dropout combined with max-norm regularization gives the lowest generalization error.

7. Salient Features

The experiments described in the previous section provide strong evidence that dropout is a useful technique for improving neural networks. In this section, we closely examine how dropout affects a neural network. We analyze the effect of dropout on the quality of features produced. We see how dropout affects the sparsity of hidden unit activations. We

方法	代码质量指标（位数）
神经网络（采用提前停止机制）（熊等人，2011年）	440
回归分析、主成分分析（熊等人，2011年）	463
支持向量机、主成分分析（熊等人，2011年）	487
采用丢弃机制的神经网络	567
贝叶斯神经网络（熊等人，2011年）	623

表8：可变剪接数据集上的实验结果。

不同条件下特定组织中可变剪接的发生情况，对于理解多种人类疾病而言具有重要意义。基于这些RNA特征，该任务的目标是预测生物学家关注的三种与剪接相关的事件的概率。评估指标为代码质量指标，它用于衡量目标概率分布与预测概率分布之间的负KL散度，数值越高表示性能越好。附录B.6中对该数据集以及这一性能指标有详细的说明。

表8汇总了不同模型在该数据集上的表现。熊等人（2011年）曾使用贝叶斯神经网络来完成这项任务。正如预期，我们发现贝叶斯神经网络的效果优于带丢弃机制的神经网络。不过，带丢弃机制的神经网络也能显著提升标准神经网络的性能，并且其表现优于其他所有方法。由于训练集规模较小，该数据集面临的主要挑战便是防止过拟合。防止过拟合的一种方法是通过主成分分析降低输入数据的维度，之后再运用支持向量机或逻辑回归等传统技术。而借助丢弃机制，我们无需进行维度降维即可避免过拟合。与贝叶斯网络仅有几十个隐藏单元相比，带丢弃机制的神经网络拥有大量隐藏单元（多达数千个），这一差异充分体现了丢弃机制强大的正则化作用。

6.5 与标准正则化方法的比较

为防止神经网络出现过拟合，人们提出了多种正则化方法，包括L2权重衰减（更广义而言即为季霍诺夫正则化，Tikhonov, 1943）、套索回归（Tibshirani, 1996）、KL稀疏性正则化以及最大范数正则化。丢弃机制也可被视为另一种用于对神经网络进行正则化的方法。在本节中，我们将利用MNIST数据集，把丢弃机制与这些正则化方法中的部分进行对比。

采用相同的包含修正线性单元的784-1024-1024-2048-10结构网络，通过随机梯度下降算法并结合不同的正则化方式对其进行了训练。表9展示了相应的实验结果。各类正则化方法所对应的各种超参数值——如衰减常数、目标稀疏度、丢弃率以及最大范数上界——都是通过验证集来确定的。我们的研究表明，将丢弃机制与最大范数正则化相结合，能够获得最低的泛化误差。

7. 核心特征

上一节中介绍的实验为丢弃机制是一种能有效提升神经网络性能的实用技术提供了有力证据。在本节中，我们将深入探讨丢弃机制对神经网络的具体影响，分析它对特征生成质量的作用，以及它如何改变隐藏单元激活信号的稀疏性。

Method	Test Classification error %
L2	1.62
L2 + L1 applied towards the end of training	1.60
L2 + KL-sparsity	1.55
Max-norm	1.35
Dropout + L2	1.25
Dropout + Max-norm	1.05

Table 9: Comparison of different regularization methods on MNIST.

also see how the advantages obtained from dropout vary with the probability of retaining units, size of the network and the size of the training set. These observations give some insight into why dropout works so well.

7.1 Effect on Features

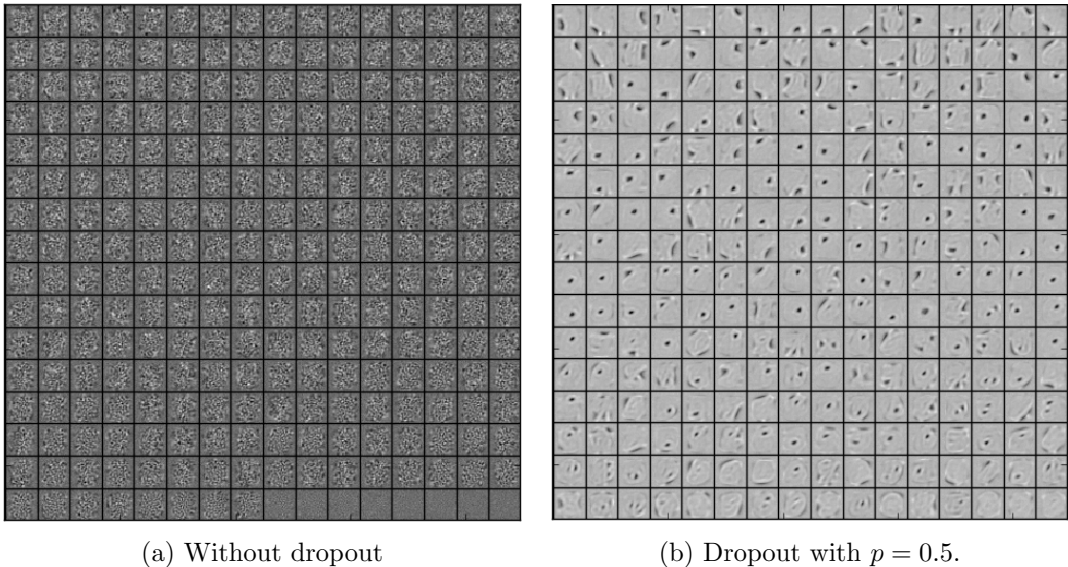


Figure 7: Features learned on MNIST with one hidden layer autoencoders having 256 rectified linear units.

In a standard neural network, the derivative received by each parameter tells it how it should change so the final loss function is reduced, *given* what all other units are doing. Therefore, units may change in a way that they fix up the mistakes of the other units. This may lead to complex co-adaptations. This in turn leads to overfitting because these co-adaptations do not generalize to unseen data. We hypothesize that for each hidden unit, dropout prevents co-adaptation by making the presence of other hidden units unreliable. Therefore, a hidden unit cannot rely on other specific units to correct its mistakes. It must perform well in a wide variety of different contexts provided by the other hidden units. To observe this effect directly, we look at the first level features learned by neural networks trained on visual tasks with and without dropout.

方法	测试分类错误率
L2	1.62
在训练接近尾声时应用L2 + L1正则化	1.60
L2 + KL-稀疏性正则化	1.55
最大范数正则化	1.35
丢弃机制结合L + L2正则化	1.25
丢弃机制 + 最大范数正则化	1.05

表9：不同正则化方法在MNIST数据集上的对比。

此外，我们还会研究丢弃机制所带来的优势如何随保留单元的概率、网络规模以及训练集大小而变化。这些观察结果有助于我们理解为何丢弃机制能发挥如此出色的效果。

7.1 对特征的影响

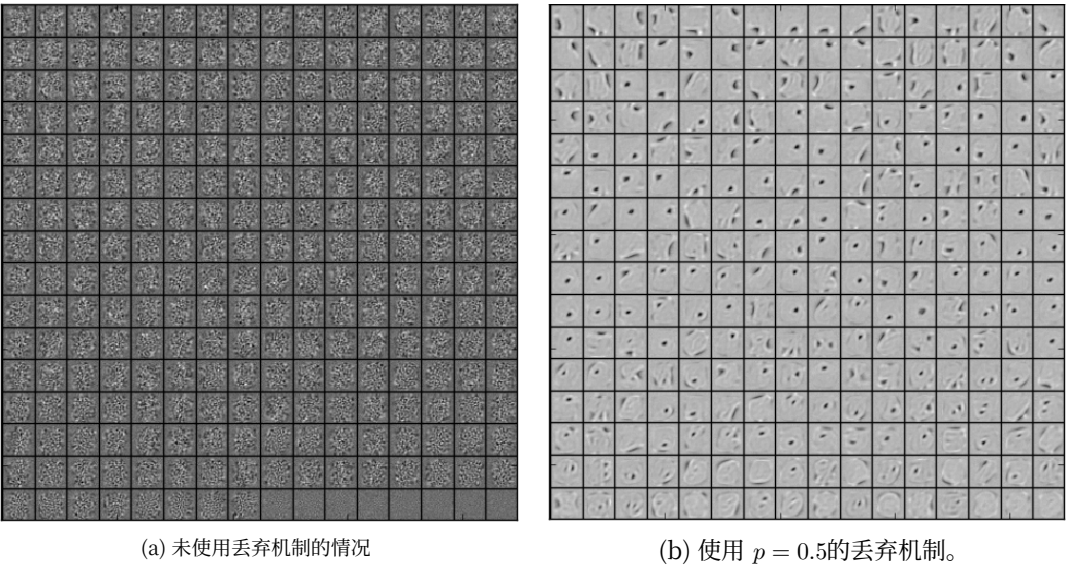


图7：在仅包含一个隐藏层的MNIST数据集自编码器上学习得到的特征，这些自编码器的每个隐藏层拥有256个线性单元。 (a) 未使用丢弃机制的情况 (b) 使用 $p = 0.5$ 的丢弃机制。

在标准神经网络中，每个参数所获得的导数会指示它应当如何变化才能降低最终的损失函数，具体变化方式取决于其他所有单元的表现。因此，这些单元可能会以某种方式相互调整，从而弥补其他单元的错误，进而引发复杂的协同适应现象。而这类协同适应由于无法泛化到未见过的数据上，最终会导致过拟合。我们假设，对于每一个隐藏单元而言，丢弃机制通过使其他隐藏单元的存在变得不可靠，从而防止了协同适应的发生。这样一来，某个隐藏单元就无法依赖特定的其他单元来纠正自身的错误，而必须在由其他隐藏单元所提供的各种不同场景下依然保持良好的性能。为了直接观察这一效应，我们研究了在带有及不带丢弃机制的情况下、针对视觉任务训练得到的神经网络所学习的第一层级特征。

Figure 7a shows features learned by an autoencoder on MNIST with a single hidden layer of 256 rectified linear units without dropout. Figure 7b shows the features learned by an identical autoencoder which used dropout in the hidden layer with $p = 0.5$. Both autoencoders had similar test reconstruction errors. However, it is apparent that the features shown in Figure 7a have co-adapted in order to produce good reconstructions. Each hidden unit on its own does not seem to be detecting a meaningful feature. On the other hand, in Figure 7b, the hidden units seem to detect edges, strokes and spots in different parts of the image. This shows that dropout does break up co-adaptations, which is probably the main reason why it leads to lower generalization errors.

7.2 Effect on Sparsity

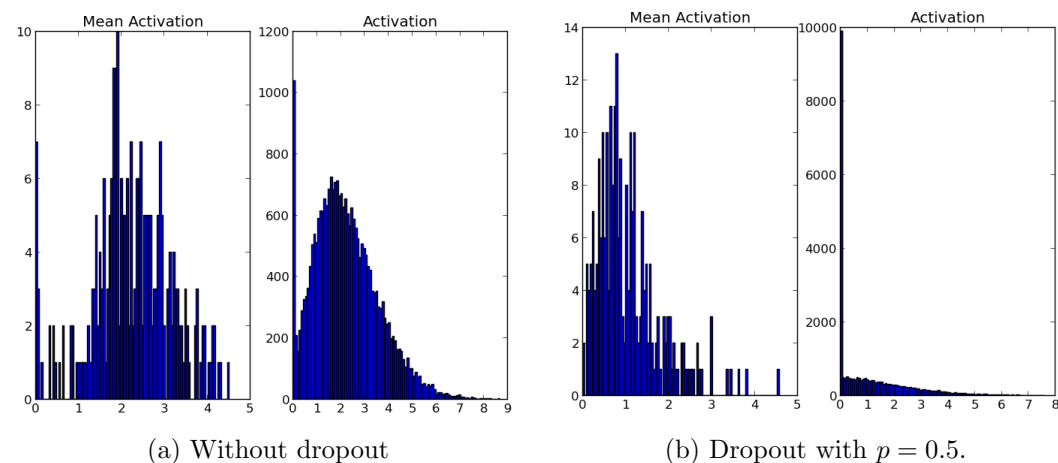


Figure 8: Effect of dropout on sparsity. ReLUs were used for both models. **Left:** The histogram of mean activations shows that most units have a mean activation of about 2.0. The histogram of activations shows a huge mode away from zero. Clearly, a large fraction of units have high activation. **Right:** The histogram of mean activations shows that most units have a smaller mean mean activation of about 0.7. The histogram of activations shows a sharp peak at zero. Very few units have high activation.

We found that as a side-effect of doing dropout, the activations of the hidden units become sparse, even when no sparsity inducing regularizers are present. Thus, dropout automatically leads to sparse representations. To observe this effect, we take the autoencoders trained in the previous section and look at the sparsity of hidden unit activations on a random mini-batch taken from the test set. Figure 8a and Figure 8b compare the sparsity for the two models. In a good sparse model, there should only be a few highly activated units for any data case. Moreover, the average activation of any unit across data cases should be low. To assess both of these qualities, we plot two histograms for each model. For each model, the histogram on the left shows the distribution of mean activations of hidden units across the minibatch. The histogram on the right shows the distribution of activations of the hidden units.

Comparing the histograms of activations we can see that fewer hidden units have high activations in Figure 8b compared to Figure 8a, as seen by the significant mass away from

图7a展示了在不含丢弃机制、仅包含256个修正线性单元的单层隐藏结构下，基于MNIST数据集训练的自编码器所学习到的特征。图7b则显示了在相同架构的自编码器中加入丢弃机制且 $p = 0.5$ 时的特征学习结果。这两款自编码器的测试重建误差相当接近。不过可以明显看出，图7a中的特征出现了协同适应现象，以此来实现较高的重建质量——单个隐藏单元似乎都无法独立检测出有意义的特征。而在图7b中，各个隐藏单元似乎能够识别图像不同区域中的边缘、线条和斑点。这表明丢弃机制确实打破了协同适应现象，而这很可能是其能够降低泛化误差的主要原因。

7.2 对稀疏性的影响

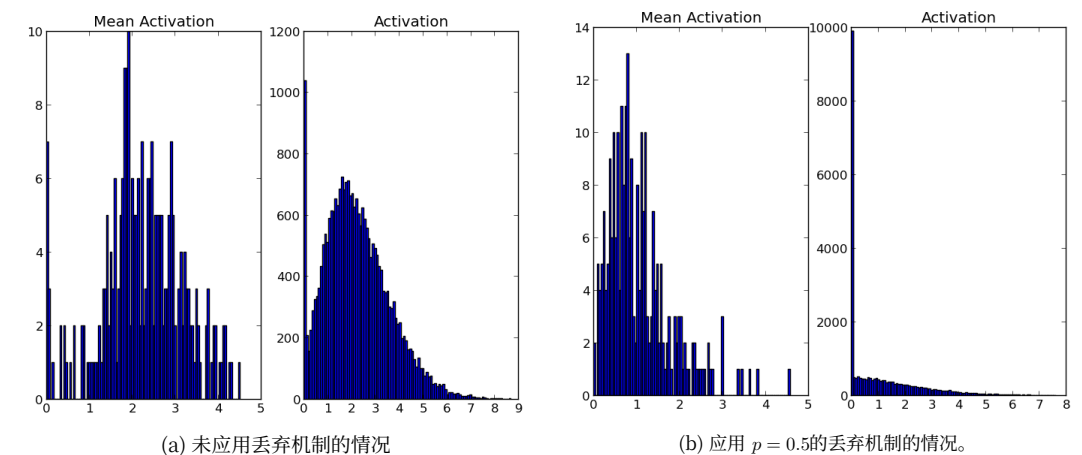


图8：丢弃机制对稀疏性的影响。两种模型均采用了ReLU激活函数。**左侧：**平均激活值的直方图显示，大多数单元的平均激活值约为2.0；而激活值本身的直方图则显示出远离零点的显著峰值，这说明有相当比例的单元具有较高的激活水平。**右侧：**平均激活值的直方图显示，大多数单元的平均激活值降至约0.7；激活值直方图则在零点处出现尖锐峰值，仅有极少数单元的激活值较高。

我们发现，即便在没有引入任何能够提升稀疏性的正则化手段时，应用丢弃机制也会使隐藏单元的激活值变得较为稀疏。因此，丢弃机制能够自动实现特征的稀疏表示。为了观察这一现象，我们使用了前一节中训练好的自编码器，随机从测试集中选取一个小批量数据，来分析隐藏单元激活值的稀疏程度。图8a和图8b分别展示了两种模型的稀疏性对比。在理想的稀疏模型中，无论面对何种数据，仅有少数单元的激活值会非常高，且所有单元在各类数据下的平均激活值都应处于较低水平。为评估这两个指标，我们为每种模型绘制了两组直方图：左侧的直方图展示的是小批量数据中各隐藏单元平均激活值的分布情况，右侧的直方图则展示的是这些单元激活值本身的分布情况。

通过对比激活值的直方图可以发现，如图8b所示，与图8a相比，具有高激活值的隐藏单元数量更少，这一差异从远离零点的显著数值集中可以看出。

zero for the net that does not use dropout. The mean activations are also smaller for the dropout net. The overall mean activation of hidden units is close to 2.0 for the autoencoder without dropout but drops to around 0.7 when dropout is used.

7.3 Effect of Dropout Rate

Dropout has a tunable hyperparameter p (the probability of retaining a unit in the network). In this section, we explore the effect of varying this hyperparameter. The comparison is done in two situations.

1. The number of hidden units is held constant.
2. The number of hidden units is changed so that the expected number of hidden units that will be retained after dropout is held constant.

In the first case, we train the same network architecture with different amounts of dropout. We use a 784-2048-2048-2048-10 architecture. No input dropout was used. Figure 9a shows the test error obtained as a function of p . If the architecture is held constant, having a small p means very few units will turn on during training. It can be seen that this has led to underfitting since the training error is also high. We see that as p increases, the error goes down. It becomes flat when $0.4 \leq p \leq 0.8$ and then increases as p becomes close to 1.

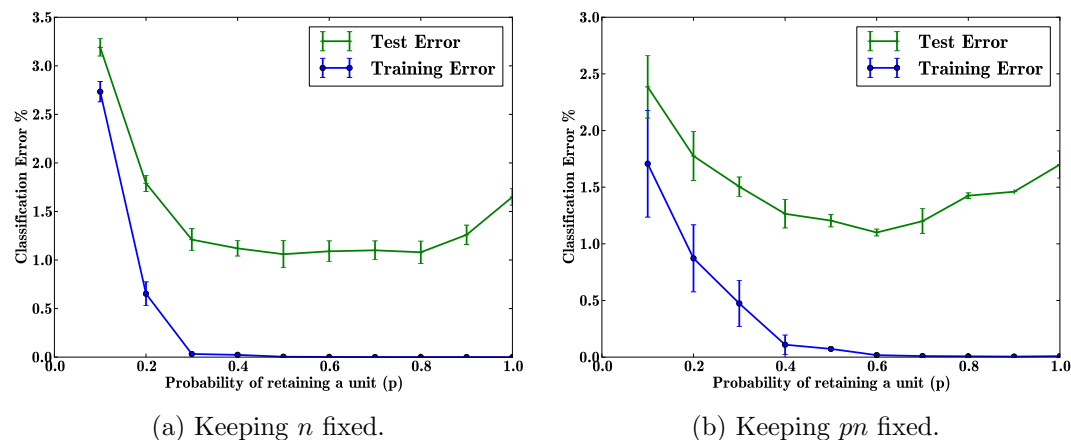


Figure 9: Effect of changing dropout rates on MNIST.

Another interesting setting is the second case in which the quantity pn is held constant where n is the number of hidden units in any particular layer. This means that networks that have small p will have a large number of hidden units. Therefore, after applying dropout, the expected number of units that are present will be the same across different architectures. However, the test networks will be of different sizes. In our experiments, we set $pn = 256$ for the first two hidden layers and $pn = 512$ for the last hidden layer. Figure 9b shows the test error obtained as a function of p . We notice that the magnitude of errors for small values of p has reduced by a lot compared to Figure 9a (for $p = 0.1$ it fell from 2.7% to 1.7%). Values of p that are close to 0.6 seem to perform best for this choice of pn but our usual default value of 0.5 is close to optimal.

对于未采用丢弃机制的神经网络，其平均激活值也更低。而带丢弃机制的神经网络的平均激活值同样更小。在没有使用丢弃机制的自编码器中，隐藏单元的整体平均激活值接近2.0，但一旦启用丢弃机制，该数值会降至约0.7左右。

7.3 丢弃率的影响

丢弃机制是一种可调节的超参数 p （即决定网络中某个单元是否会被保留的概率）。在本节中，我们将探讨调整这一超参数所带来的影响，并在两种不同的情况下进行对比分析。

1. 保持隐藏单元的数量不变；2. 调整隐藏单元的数量，以确保在应用丢弃机制后仍保留的隐藏单元预期数量保持不变。

在第一种情况下，我们使用相同的网络架构但设置不同的丢弃率来训练模型。此处采用的架构为784-2048-2048-2048-10结构，且未启用输入层的丢弃机制。图9a展示了作为 p 函数变化的测试误差情况。当网络架构保持不变时，较小的 p 值意味着在训练过程中仅有极少的单元会被激活。由此可见，这种情况会导致模型欠拟合，因为训练误差也会很高。随着 p 的增加，误差会逐渐下降；当 $0.4 \leq p \leq 0.8$ 时误差趋于平稳，而当 p 接近1时误差又会开始上升。

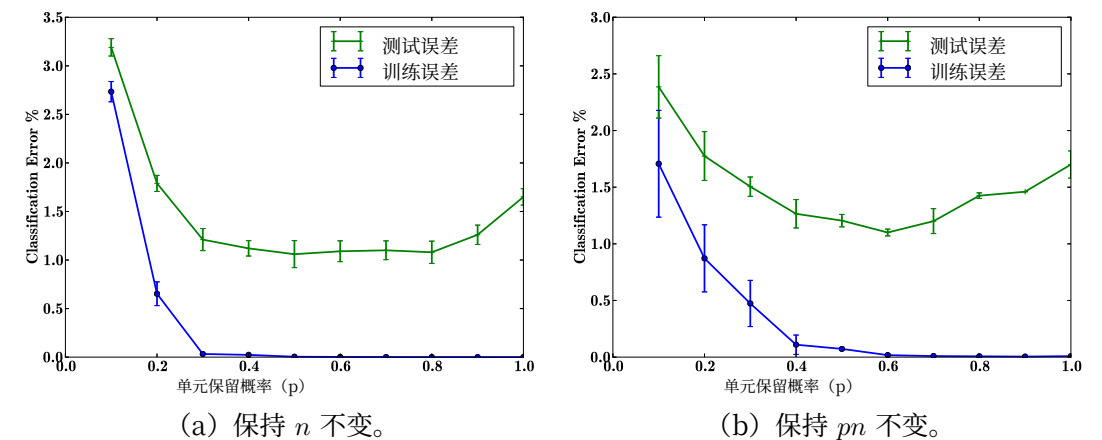


图9: MNIST数据集上不同丢弃率所造成的影响。

另一个有趣的设定是第二种情况：此时 pn 保持不变，而 n 则表示任意特定层中的隐藏单元数量。这意味着 p 较小的神经网络将拥有较多的隐藏单元。因此，在应用丢弃机制后，不同网络架构中存活的单元预期数量将会相同，不过测试网络的规模则会有所差异。在我们的实验中，前两层隐藏层分别设置为 $pn = 256$ ，最后一层隐藏层设置为 $pn = 512$ 。图9b展示了以 p 为变量时的测试误差变化情况。我们可以发现，当 p 取较小值时，其误差幅度相比图9a有了显著下降（在 $p = 0.1$ 时，误差从2.7%降至1.7%）。接近0.6的 p 值似乎在这种 pn 设置下表现最佳，不过我们通常使用的默认值0.5也已经非常接近最优值了。

7.4 Effect of Data Set Size

One test of a good regularizer is that it should make it possible to get good generalization error from models with a large number of parameters trained on small data sets. This section explores the effect of changing the data set size when dropout is used with feed-forward networks. Huge neural networks trained in the standard way overfit massively on small data sets. To see if dropout can help, we run classification experiments on MNIST and vary the amount of data given to the network.

The results of these experiments are shown in Figure 10. The network was given data sets of size 100, 500, 1K, 5K, 10K and 50K chosen randomly from the MNIST training set. The same network architecture (784-1024-1024-2048-10) was used for all data sets. Dropout with $p = 0.5$ was performed at all the hidden layers and $p = 0.8$ at the input layer. It can be observed that for extremely small data sets (100, 500) dropout does not give any improvements. The model has enough parameters that it can overfit on the training data, even with all the noise coming from dropout. As the size of the data set is increased, the gain from doing dropout increases up to a point and then declines. This suggests that for any given architecture and dropout rate, there is a “sweet spot” corresponding to some amount of data that is large enough to not be memorized in spite of the noise but not so large that overfitting is not a problem anyways.

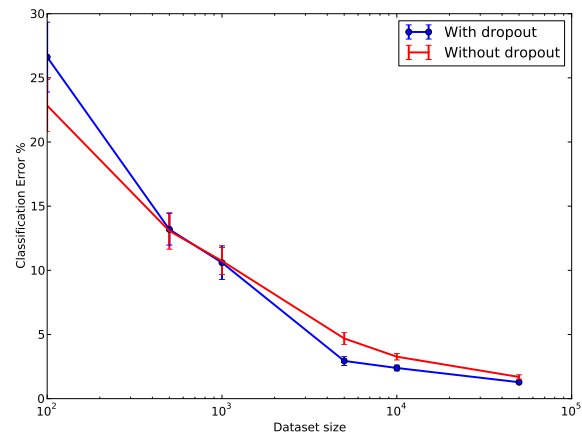


Figure 10: Effect of varying data set size.

7.5 Monte-Carlo Model Averaging vs. Weight Scaling

The efficient test time procedure that we propose is to do an approximate model combination by scaling down the weights of the trained neural network. An expensive but more correct way of averaging the models is to sample k neural nets using dropout for each test case and average their predictions. As $k \rightarrow \infty$, this Monte-Carlo model average gets close to the true model average. It is interesting to see empirically how many samples k are needed to match the performance of the approximate averaging method. By computing the error for different values of k we can see how quickly the error rate of the finite-sample average approaches the error rate of the true model average.

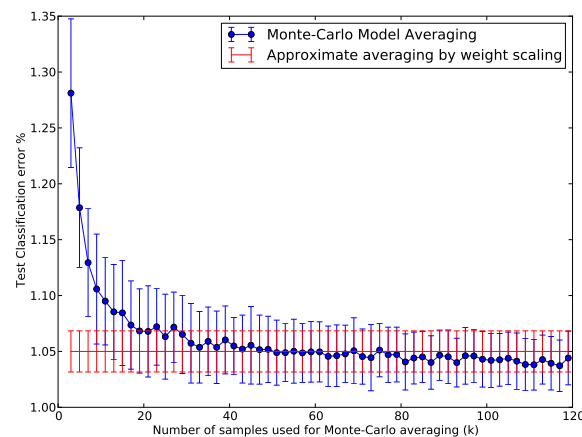


Figure 11: Monte-Carlo model averaging vs. weight scaling.

7.4 数据集规模的影响

判断一个优秀的正则化手段的一个标准是，它应当能够让那些在小型数据集上训练、且拥有大量参数的模型依然能够获得较低的泛化误差。本节将探讨在反馈前神经网络中使用丢弃机制时，数据集规模发生变化所带来的影响。按照常规方法训练的大型神经网络在面对小型数据集时往往会出现严重的过拟合现象。为了验证丢弃机制是否有助于解决这一问题，我们在MNIST数据集上开展了分类实验，并改变了输入网络的数据量。

这些实验的结果如图10所示。该网络被提供了从MNIST训练集中随机选取的规模为100、500、1K、5K、10K和50K的数据集，所有数据集均采用相同的架构（784-1024-1024-2048-10）。在所有的隐藏层都应用了 $p = 0.5$ 的丢弃机制，而在输入层则应用了 $p = 0.8$ 的机制。可以看出，当数据集规模极小（100、500）时，丢弃机制并不会带来任何改进——由于模型拥有足够的参数，即便存在来自丢弃机制的噪声，它依然可能对训练数据产生过拟合。而随着数据集规模的增大，其优势也会逐渐显现。

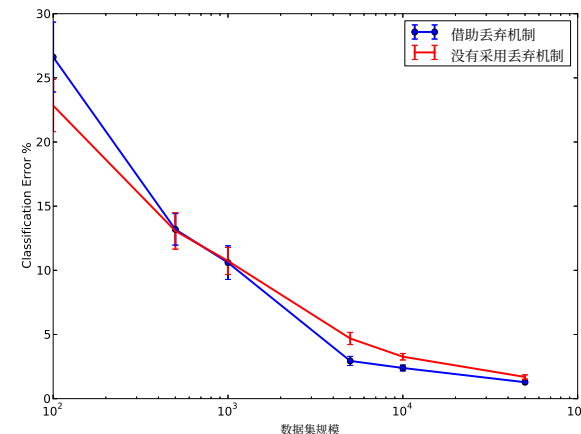


图10：不同数据集规模带来的影响。

随着丢弃机制的运用，其效果会在某一临界点之前持续提升，之后则开始下降。这一现象表明，对于任何特定的网络架构和丢弃率而言，都存在一个“最佳区间”——即数据量足够大，足以让模型在存在噪声的情况下仍不会发生记忆效应，但又不会大到导致过拟合问题出现。

7.5 蒙特卡洛模型平均法与权重缩放的对比

我们提出的高效测试阶段处理方法是通过对已训练的神经网络的权重进行缩放，从而实现模型的近似组合。另一种虽然计算成本更高但精度更高的平均模型方法则是：针对每个测试案例，利用丢弃机制随机选取 k 个神经网络，再对其预测结果取平均。正如 $k \rightarrow \infty$ 所示，这种蒙特卡洛模型平均值能够逐渐逼近真实模型平均值。通过实证研究来探究需要多少个样本 k 才能达到近似平均法同样的性能，颇具意义。通过计算不同 k 值下的误差，我们可以直观地看到有限样本平均法的误差率会多快接近真实模型平均法的误差率。

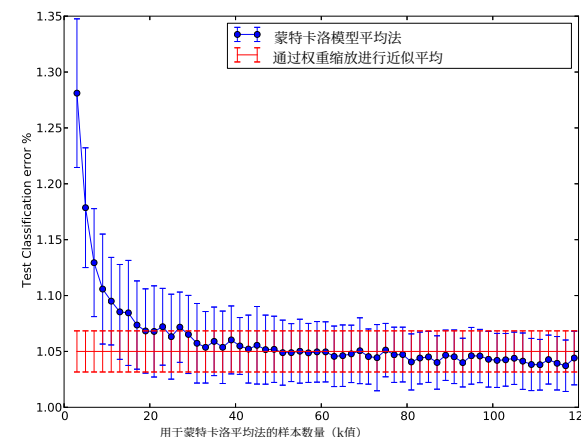


图11：蒙特卡洛模型平均法与权重缩放的对比。

We again use the MNIST data set and do classification by averaging the predictions of k randomly sampled neural networks. Figure 11 shows the test error rate obtained for different values of k . This is compared with the error obtained using the weight scaling method (shown as a horizontal line). It can be seen that around $k = 50$, the Monte-Carlo method becomes as good as the approximate method. Thereafter, the Monte-Carlo method is slightly better than the approximate method but well within one standard deviation of it. This suggests that the weight scaling method is a fairly good approximation of the true model average.

8. Dropout Restricted Boltzmann Machines

Besides feed-forward neural networks, dropout can also be applied to Restricted Boltzmann Machines (RBM). In this section, we formally describe this model and show some results to illustrate its key properties.

8.1 Model Description

Consider an RBM with visible units $\mathbf{v} \in \{0, 1\}^D$ and hidden units $\mathbf{h} \in \{0, 1\}^F$. It defines the following probability distribution

$$P(\mathbf{h}, \mathbf{v}; \theta) = \frac{1}{\mathcal{Z}(\theta)} \exp(\mathbf{v}^\top W \mathbf{h} + \mathbf{a}^\top \mathbf{h} + \mathbf{b}^\top \mathbf{v}).$$

Where $\theta = \{W, \mathbf{a}, \mathbf{b}\}$ represents the model parameters and $\mathcal{Z}$ is the partition function.

Dropout RBMs are RBMs augmented with a vector of binary random variables $\mathbf{r} \in \{0, 1\}^F$. Each random variable r_j takes the value 1 with probability p , independent of others. If r_j takes the value 1, the hidden unit h_j is retained, otherwise it is dropped from the model. The joint distribution defined by a Dropout RBM can be expressed as

$$\begin{aligned} P(\mathbf{r}, \mathbf{h}, \mathbf{v}; p, \theta) &= P(\mathbf{r}; p) P(\mathbf{h}, \mathbf{v} | \mathbf{r}; \theta), \\ P(\mathbf{r}; p) &= \prod_{j=1}^F p^{r_j} (1-p)^{1-r_j}, \\ P(\mathbf{h}, \mathbf{v} | \mathbf{r}; \theta) &= \frac{1}{\mathcal{Z}'(\theta, \mathbf{r})} \exp(\mathbf{v}^\top W \mathbf{h} + \mathbf{a}^\top \mathbf{h} + \mathbf{b}^\top \mathbf{v}) \prod_{j=1}^F g(h_j, r_j), \\ g(h_j, r_j) &= \mathbb{1}(r_j = 1) + \mathbb{1}(r_j = 0) \mathbb{1}(h_j = 0). \end{aligned}$$

$\mathcal{Z}'(\theta, \mathbf{r})$ is the normalization constant. $g(h_j, r_j)$ imposes the constraint that if $r_j = 0$, h_j must be 0. The distribution over $\mathbf{h}$, conditioned on $\mathbf{v}$ and $\mathbf{r}$ is factorial

$$\begin{aligned} P(\mathbf{h} | \mathbf{r}, \mathbf{v}) &= \prod_{j=1}^F P(h_j | r_j, \mathbf{v}), \\ P(h_j = 1 | r_j, \mathbf{v}) &= \mathbb{1}(r_j = 1) \sigma \left(b_j + \sum_i W_{ij} v_i \right). \end{aligned}$$

我们依旧使用MNIST数据集，并通过对 k 个随机抽取的神经网络的预测结果进行平均来实现分类任务。图11展示了不同 k 值对应的测试误差率，同时还将该结果与采用权重缩放方法得到的误差率（以水平线形式呈现）进行了对比。可以看出，在 $k = 50$ 左右时，蒙特卡洛方法的性能已经与近似平均法相当；此后，蒙特卡洛方法的性能虽略优于近似平均法，但其差异仍在后者一个标准差范围内。这一现象表明，权重缩放方法实际上已经是对真实模型平均值相当不错的近似方式。

8. 丢弃机制与受限玻尔兹曼机

除了前馈神经网络之外，丢弃机制也可应用于受限玻尔兹曼机（RBM）。在本节中，我们将正式阐述该模型，并通过一些实验结果来展示其核心特性。

8.1 模型描述

考虑一个具有可见单元 $\mathbf{v} \in \{0, 1\}^D$ 和隐藏单元 $\mathbf{h} \in \{0, 1\}^F$ 的RBM，它定义了如下的概率分布。

$$P(\mathbf{h}, \mathbf{v}; \theta) = \frac{1}{\mathcal{Z}(\theta)} \exp(\mathbf{v}^\top W \mathbf{h} + \mathbf{a}^\top \mathbf{h} + \mathbf{b}^\top \mathbf{v}).$$

在何处 $\theta = \{W, \mathbf{a}, \mathbf{b}\}$ 代表模型参数，而 $\mathcal{Z}$ 则为配分函数。

带有丢弃机制的受限玻尔兹曼机是在普通受限玻尔兹曼机的基础上增加了一组二进制随机变量 $\mathbf{r} \in \{0, 1\}^F$ 而构成的。其中每个随机变量 r_j 以 p 的概率取值1，且这些变量的取值相互独立。如果 r_j 的值为1，则对应的隐藏单元 h_j 会被保留下来，否则就会从模型中剔除。由这类机器所定义的联合分布可以表示为

$$\begin{aligned} P(\mathbf{r}, \mathbf{h}, \mathbf{v}; p, \theta) &= P(\mathbf{r}; p) P(\mathbf{h}, \mathbf{v} | \mathbf{r}; \theta), \\ P(\mathbf{r}; p) &= \prod_{j=1}^F p^{r_j} (1-p)^{1-r_j}, \\ P(\mathbf{h}, \mathbf{v} | \mathbf{r}; \theta) &= \frac{1}{\mathcal{Z}'(\theta, \mathbf{r})} \exp(\mathbf{v}^\top W \mathbf{h} + \mathbf{a}^\top \mathbf{h} + \mathbf{b}^\top \mathbf{v}) \prod_{j=1}^F g(h_j, r_j), \\ g(h_j, r_j) &= \mathbb{1}(r_j = 1) + \mathbb{1}(r_j = 0) \mathbb{1}(h_j = 0). \end{aligned}$$

$\mathcal{Z}'(\theta, \mathbf{r})$ 即为归一化常数。 $g(h_j, r_j)$ 设定了这样的约束：若 $r_j = 0$ ，则 h_j 必须为0。在给定 $\mathbf{v}$ 和 $\mathbf{r}$ 的条件下， $\mathbf{h}$ 的分布属于阶乘分布类型。

$$\begin{aligned} P(\mathbf{h} | \mathbf{r}, \mathbf{v}) &= \prod_{j=1}^F P(h_j | r_j, \mathbf{v}), \\ P(h_j = 1 | r_j, \mathbf{v}) &= \mathbb{1}(r_j = 1) \sigma \left(b_j + \sum_i W_{ij} v_i \right). \end{aligned}$$

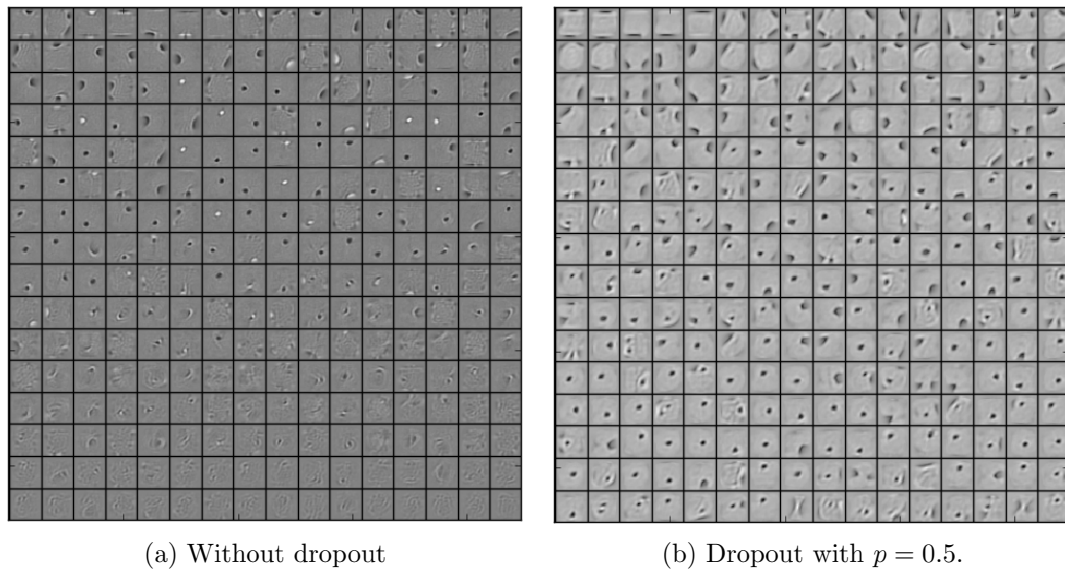


Figure 12: Features learned on MNIST by 256 hidden unit RBMs. The features are ordered by L2 norm.

The distribution over $\mathbf{v}$ conditioned on $\mathbf{h}$ is same as that of an RBM

$$P(\mathbf{v}|\mathbf{h}) = \prod_{i=1}^D P(v_i|\mathbf{h}),$$

$$P(v_i = 1|\mathbf{h}) = \sigma \left(a_i + \sum_j W_{ij} h_j \right).$$

Conditioned on $\mathbf{r}$, the distribution over $\{\mathbf{v}, \mathbf{h}\}$ is same as the distribution that an RBM would impose, except that the units for which $r_j = 0$ are dropped from $\mathbf{h}$. Therefore, the Dropout RBM model can be seen as a mixture of exponentially many RBMs with shared weights each using a different subset of $\mathbf{h}$.

8.2 Learning Dropout RBMs

Learning algorithms developed for RBMs such as Contrastive Divergence (Hinton et al., 2006) can be directly applied for learning Dropout RBMs. The only difference is that $\mathbf{r}$ is first sampled and only the hidden units that are retained are used for training. Similar to dropout neural networks, a different $\mathbf{r}$ is sampled for each training case in every minibatch. In our experiments, we use CD-1 for training dropout RBMs.

8.3 Effect on Features

Dropout in feed-forward networks improved the quality of features by reducing co-adaptations. This section explores whether this effect transfers to Dropout RBMs as well.

Figure 12a shows features learned by a binary RBM with 256 hidden units. Figure 12b shows features learned by a dropout RBM with the same number of hidden units. Features

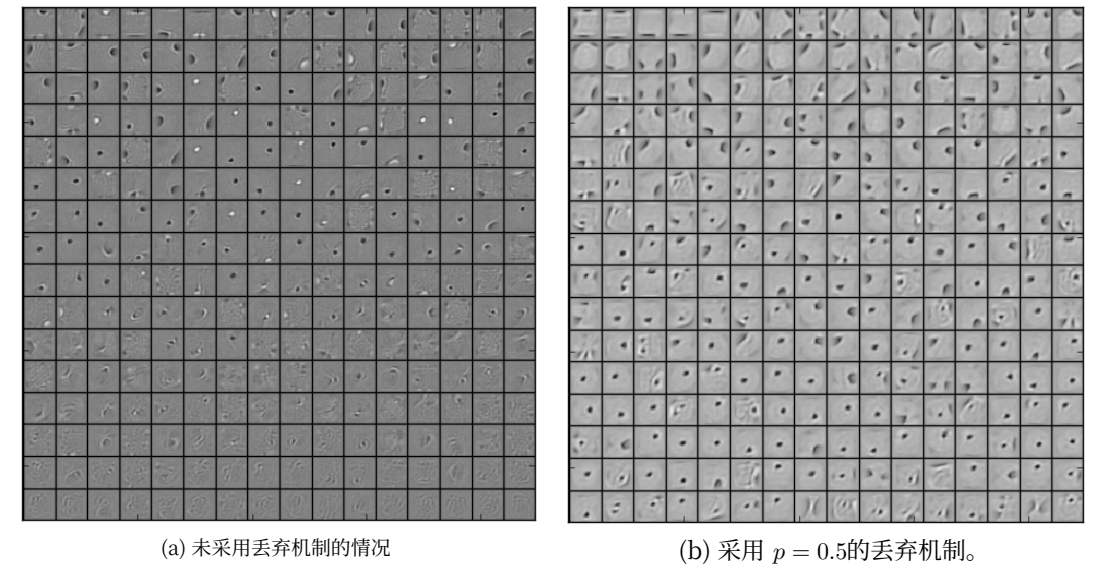


图12: 由包含256个隐藏单元的随机玻尔兹曼机在MNIST数据集上学习得到的特征。这些特征是按照L2范数进行排序的。

在给定 $\mathbf{h}$ 的条件下, $\mathbf{v}$ 的分布与随机玻尔兹曼机的分布相同。

$$P(\mathbf{v}|\mathbf{h}) = \prod_{i=1}^D P(v_i|\mathbf{h}),$$

$$P(v_i = 1|\mathbf{h}) = \sigma \left(a_i + \sum_j W_{ij} h_j \right).$$

在 $\mathbf{r}$ 为条件的情况下, $\{\mathbf{v}, \mathbf{h}\}$ 所对应的分布与普通随机玻尔兹曼机会产生的分布相同, 不同之处在于那些 $r_j = 0$ 被从 $\mathbf{h}$ 中移除的单元。因此, 带丢弃机制的受限玻尔兹曼机模型可被视为由数量呈指数级增长的多个共享权重的随机玻尔兹曼机混合而成, 而这些随机玻尔兹曼机各自使用 $\mathbf{h}$ 的不同子集。

8.2 学习带丢弃机制的受限玻尔兹曼机模型

为受限玻尔兹曼机开发的各类学习算法, 例如对比散度算法 (Hinton等人, 2006年), 可直接用于训练带有丢弃机制的受限玻尔兹曼机。二者唯一的区别在于, 首先需要对 $\mathbf{r}$ 进行采样, 随后仅使用那些被保留下来的隐藏单元参与训练。与丢弃神经网络类似, 每个小批量中的每一个训练案例都会抽取不同的 $\mathbf{r}$ 值。在我们的实验中, 我们采用了CD-1算法来训练这类带有丢弃机制的受限玻尔兹曼机。

8.3 对特征的影响

在前馈网络中, 丢弃机制通过减少协同适应现象提升了特征的质量。本节将探讨这种效果是否同样适用于带有丢弃机制的受限玻尔兹曼机。

图12a展示了具有256个隐藏单位的二进制随机玻尔兹曼机所学习到的特征。图12b则展示了拥有相同数量隐藏单元的采用丢弃机制的随机玻尔兹曼机所学习到的特征。这些特征

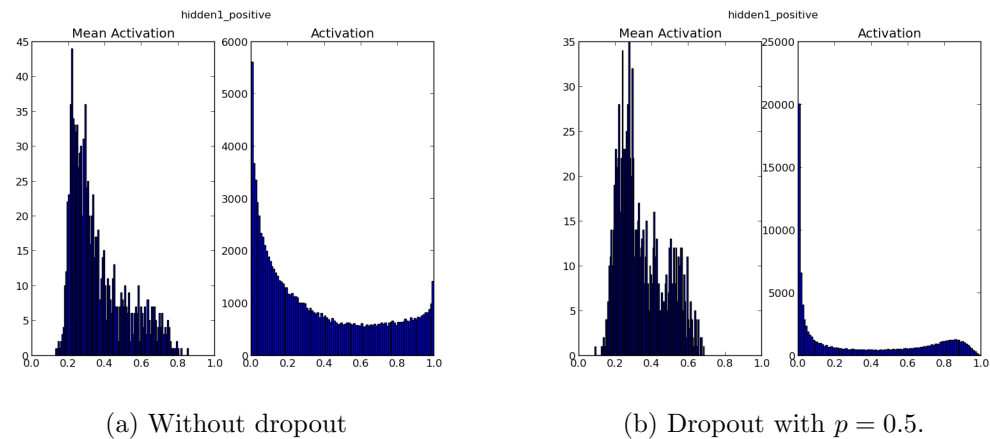


Figure 13: Effect of dropout on sparsity. **Left:** The activation histogram shows that a large number of units have activations away from zero. **Right:** A large number of units have activations close to zero and very few units have high activation.

learned by the dropout RBM appear qualitatively different in the sense that they seem to capture features that are coarser compared to the sharply defined stroke-like features in the standard RBM. There seem to be very few dead units in the dropout RBM relative to the standard RBM.

8.4 Effect on Sparsity

Next, we investigate the effect of dropout RBM training on sparsity of the hidden unit activations. Figure 13a shows the histograms of hidden unit activations and their means on a test mini-batch after training an RBM. Figure 13b shows the same for dropout RBMs. The histograms clearly indicate that the dropout RBMs learn much sparser representations than standard RBMs even when no additional sparsity inducing regularizer is present.

9. Marginalizing Dropout

Dropout can be seen as a way of adding noise to the states of hidden units in a neural network. In this section, we explore the class of models that arise as a result of marginalizing this noise. These models can be seen as deterministic versions of dropout. In contrast to standard (“Monte-Carlo”) dropout, these models do not need random bits and it is possible to get gradients for the marginalized loss functions. In this section, we briefly explore these models.

Deterministic algorithms have been proposed that try to learn models that are robust to feature deletion at test time (Globerson and Roweis, 2006). Marginalization in the context of denoising autoencoders has been explored previously (Chen et al., 2012). The marginalization of dropout noise in the context of linear regression was discussed in Srivastava (2013). Wang and Manning (2013) further explored the idea of marginalizing dropout to speed-up training. van der Maaten et al. (2013) investigated different input noise distributions and

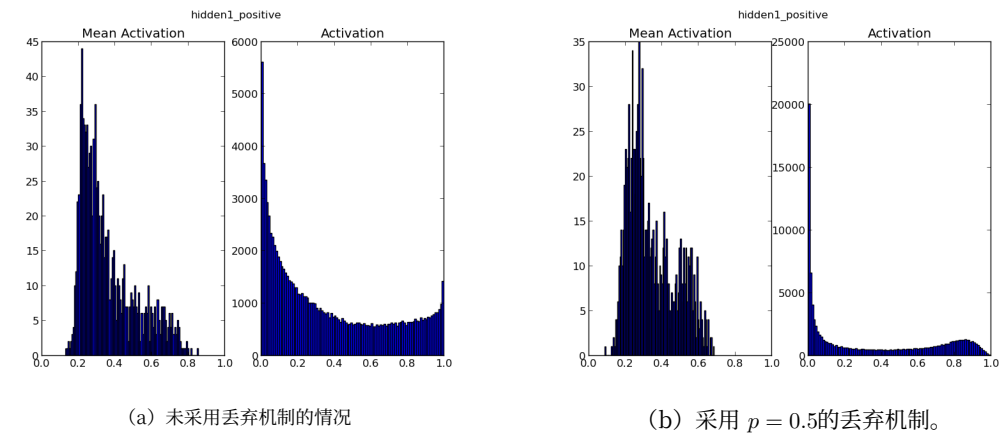


图13: 丢弃机制对稀疏性的影响。左侧所示的激活值直方图表明，大量单元的激活值远离零；而右侧则显示大量单元的激活值接近零，仅有极少数单元的激活值较高。

采用丢弃机制的随机玻尔兹曼机学习到的特征在质量上存在明显差异，它们捕捉到的特征较为粗糙，与标准随机玻尔兹曼机中那些边界清晰的笔触状特征不同。相比标准随机玻尔兹曼机，采用丢弃机制的模型中的失效单元数量要少得多。

8.4 对稀疏性的影响

接下来，我们研究在随机玻尔兹曼机训练过程中应用丢弃机制会对隐藏单元激活值的稀疏性产生何种影响。图13a展示了在完成随机玻尔兹曼机训练后，测试小批量数据中隐藏单元激活值的直方图及其均值；图13b则展示了应用了丢弃机制的随机玻尔兹曼机的对应结果。这些直方图清晰地表明，即便没有其他能够提升稀疏性的正则化手段，使用丢弃机制的随机玻尔兹曼机也能学习到比标准随机玻尔兹曼机更为稀疏的表示结构。

9. 从概率角度理解丢弃机制

丢弃机制可以被视作一种向神经网络中隐藏单元的状态添加噪声的方法。在本节中，我们将探讨通过消除这种噪声而产生的模型类别，这类模型可被视为丢弃机制的确定性版本。与传统的（“蒙特卡洛”）丢弃机制不同，这类模型无需随机数值，且能够获取对应消除噪声后的损失函数的梯度。本节将简要介绍这些模型。

已有研究提出了确定性算法，旨在学习能够在测试阶段面对特征缺失时依然保持稳健性的模型（格洛伯森与罗伊斯，2006年）。在去噪自编码器的框架下，边缘化处理方法早前就有人探讨过（陈等人，2012年）。至于线性回归场景中丢弃机制带来的噪声的边缘化问题，则在斯里瓦斯塔瓦（2013年）的著作中有相关论述。Wang与Manning（2013年）进一步研究了通过边缘化丢弃机制来加速训练的方法。范德马滕等人（2013年）则针对不同的输入噪声分布进行了研究，并探讨了由此产生的正则化项。韦杰等人（2013年）阐述了为何可以将丢弃机制视为一种自适应的正则化手段。

the regularizers obtained by marginalizing this noise. Wager et al. (2013) describes how dropout can be seen as an adaptive regularizer.

9.1 Linear Regression

First we explore a very simple case of applying dropout to the classical problem of linear regression. Let $X \in \mathbb{R}^{N \times D}$ be a data matrix of N data points. $\mathbf{y} \in \mathbb{R}^N$ be a vector of targets. Linear regression tries to find a $\mathbf{w} \in \mathbb{R}^D$ that minimizes

$$\|\mathbf{y} - X\mathbf{w}\|^2.$$

When the input X is dropped out such that any input dimension is retained with probability p , the input can be expressed as $R * X$ where $R \in \{0, 1\}^{N \times D}$ is a random matrix with $R_{ij} \sim \text{Bernoulli}(p)$ and $*$ denotes an element-wise product. Marginalizing the noise, the objective function becomes

$$\underset{\mathbf{w}}{\text{minimize}} \quad \mathbb{E}_{R \sim \text{Bernoulli}(p)} [\|\mathbf{y} - (R * X)\mathbf{w}\|^2].$$

This reduces to

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1-p)\|\Gamma\mathbf{w}\|^2,$$

where $\Gamma = (\text{diag}(X^\top X))^{1/2}$. Therefore, dropout with linear regression is equivalent, in expectation, to ridge regression with a particular form for Γ . This form of Γ essentially scales the weight cost for weight w_i by the standard deviation of the i th dimension of the data. If a particular data dimension varies a lot, the regularizer tries to squeeze its weight more.

Another interesting way to look at this objective is to absorb the factor of p into $\mathbf{w}$. This leads to the following form

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|\mathbf{y} - X\tilde{\mathbf{w}}\|^2 + \frac{1-p}{p}\|\Gamma\tilde{\mathbf{w}}\|^2,$$

where $\tilde{\mathbf{w}} = p\mathbf{w}$. This makes the dependence of the regularization constant on p explicit. For p close to 1, all the inputs are retained and the regularization constant is small. As more dropout is done (by decreasing p), the regularization constant grows larger.

9.2 Logistic Regression and Deep Networks

For logistic regression and deep neural nets, it is hard to obtain a closed form marginalized model. However, Wang and Manning (2013) showed that in the context of dropout applied to logistic regression, the corresponding marginalized model can be trained approximately. Under reasonable assumptions, the distributions over the inputs to the logistic unit and over the gradients of the marginalized model are Gaussian. Their means and variances can be computed efficiently. This approximate marginalization outperforms Monte-Carlo dropout in terms of training time and generalization performance.

However, the assumptions involved in this technique become successively weaker as more layers are added. Therefore, the results are not directly applicable to deep networks.

韦杰等人（2013年）描述了如何将丢弃机制视作一种自适应的正则化器。

9.1 线性回归分析

首先，我们探讨一个非常简单的案例，即如何将丢弃机制应用于经典的线性回归问题。设

$X \in \mathbb{R}^{N \times D}$ 为包含 N 个数据点的矩阵， $\mathbf{y} \in \mathbb{R}^N$ 则为目标值向量。线性回归的目标是找到一个 $\mathbf{w} \in \mathbb{R}^D$ ，使得某种损失函数达到最小值。

$$\|\mathbf{y} - X\mathbf{w}\|^2.$$

当对输入 X 应用丢弃机制时，每个输入维度会以 p 的概率被保留下来，此时输入可以表示为 $R * X$ ，其中 $R \in \{0, 1\}^{N \times D}$ 是一个随机矩阵，其元素均为 $R_{ij} \sim \text{Bernoulli}(p)$ 分布的随机数，而 $*$ 则表示元素级乘积。在忽略噪声因素后，对应的优化目标函数便可以确定。

$$\underset{\mathbf{w}}{\text{minimize}} \quad \mathbb{E}_{R \sim \text{Bernoulli}(p)} [\|\mathbf{y} - (R * X)\mathbf{w}\|^2].$$

这可简化为

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|\mathbf{y} - pX\mathbf{w}\|^2 + p(1-p)\|\Gamma\mathbf{w}\|^2,$$

其表达式为 $\Gamma = (\text{diag}(X^\top X))^{1/2}$ 。因此，从期望值的角度来看，将丢弃机制与线性回归结合使用，等同于采用特定形式的岭回归来处理问题，这里的 Γ 就是所对应的特定形式。这种 Γ 机制本质上是通过数据第 i 维的标准差来调整对应权重 w_i 的惩罚系数。如果某个数据维度的数值变化较大，正则化项就会试图对该维度的权重施加更大的约束。

另一种审视该目标函数的有趣方式是将 p 这一因子纳入 $\mathbf{w}$ 之中，由此便能得到如下形式。

$$\underset{\mathbf{w}}{\text{minimize}} \quad \|\mathbf{y} - X\tilde{\mathbf{w}}\|^2 + \frac{1-p}{p}\|\Gamma\tilde{\mathbf{w}}\|^2,$$

其中 $\tilde{\mathbf{w}} = p\mathbf{w}$ 。这样的表达方式清晰地体现了正则化常数与 p 之间的依赖关系。当 p 接近1时，所有输入特征都会被保留，此时正则化常数的数值也较小；而随着丢弃机制的强度增加（即 p 的值减小），正则化常数的数值则会逐渐变大。

9.2 逻辑回归与深度网络

对于逻辑回归和深度神经网络而言，很难得到具有封闭形式的边际模型。不过Wang与Manning（2013年）指出，在将丢弃机制应用于逻辑回归的背景下，相应的边际模型是可以被近似训练出来的。在合理的假设条件下，逻辑单元的输入特征及其边际模型的梯度所遵循的分布均为高斯分布，其均值和方差也能够被高效计算。相较于蒙特卡洛丢弃机制，这种近似边际化方法在训练阶段的时间消耗以及泛化性能方面都具有更优的表现。

然而，随着层数的增加，该技术所依赖的假设会逐渐变得不那么成立，因此其结果无法直接应用于深度神经网络。

Data Set	Architecture	Bernoulli dropout	Gaussian dropout
MNIST	2 layers, 1024 units each	1.08 ± 0.04	0.95 ± 0.04
CIFAR-10	3 conv + 2 fully connected layers	12.6 ± 0.1	12.5 ± 0.1

Table 10: Comparison of classification error % with Bernoulli and Gaussian dropout. For MNIST, the Bernoulli model uses $p = 0.5$ for the hidden units and $p = 0.8$ for the input units. For CIFAR-10, we use $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ going from the input layer to the top. The value of σ for the Gaussian dropout models was set to be $\sqrt{\frac{1-p}{p}}$. Results were averaged over 10 different random seeds.

10. Multiplicative Gaussian Noise

Dropout involves multiplying hidden activations by Bernoulli distributed random variables which take the value 1 with probability p and 0 otherwise. This idea can be generalized by multiplying the activations with random variables drawn from other distributions. We recently discovered that multiplying by a random variable drawn from $\mathcal{N}(1, 1)$ works just as well, or perhaps better than using Bernoulli noise. This new form of dropout amounts to adding a Gaussian distributed random variable with zero mean and standard deviation equal to the activation of the unit. That is, each hidden activation h_i is perturbed to $h_i + h_i r$ where $r \sim \mathcal{N}(0, 1)$, or equivalently $h_i r'$ where $r' \sim \mathcal{N}(1, 1)$. We can generalize this to $r' \sim \mathcal{N}(1, \sigma^2)$ where σ becomes an additional hyperparameter to tune, just like p was in the standard (Bernoulli) dropout. The expected value of the activations remains unchanged, therefore no weight scaling is required at test time.

In this paper, we described dropout as a method where we retain units with probability p at training time and scale down the weights by multiplying them by a factor of p at test time. Another way to achieve the same effect is to scale up the retained activations by multiplying by $1/p$ at training time and not modifying the weights at test time. These methods are equivalent with appropriate scaling of the learning rate and weight initializations at each layer.

Therefore, dropout can be seen as multiplying h_i by a Bernoulli random variable r_b that takes the value $1/p$ with probability p and 0 otherwise. $E[r_b] = 1$ and $Var[r_b] = (1 - p)/p$. For the Gaussian multiplicative noise, if we set $\sigma^2 = (1 - p)/p$, we end up multiplying h_i by a random variable r_g , where $E[r_g] = 1$ and $Var[r_g] = (1 - p)/p$. Therefore, both forms of dropout can be set up so that the random variable being multiplied by has the same mean and variance. However, given these first and second order moments, r_g has the highest entropy and r_b has the lowest. Both these extremes work well, although preliminary experimental results shown in Table 10 suggest that the high entropy case might work slightly better. For each layer, the value of σ in the Gaussian model was set to be $\sqrt{\frac{1-p}{p}}$ using the p from the corresponding layer in the Bernoulli model.

11. Conclusion

Dropout is a technique for improving neural networks by reducing overfitting. Standard backpropagation learning builds up brittle co-adaptations that work for the training data but do not generalize to unseen data. Random dropout breaks up these co-adaptations by

数据集	网络架构	伯努利丢弃	高斯丢弃
MNIST数据集	共2层，每层包含1024个单元	1.08 ± 0.04	0.95 ± 0.04
CIFAR-10数据集	3个卷积层以及 + 2 个全连接层	12.6 ± 0.1	12.5 ± 0.1

表10：伯努利丢弃机制与高斯丢弃机制在分类误差百分比方面的对比。对于MNIST数据集，伯努利模型在隐藏单元上使用 $p = 0.5$ ，而在输入单元上使用 $p = 0.8$ ；对于CIFAR-10数据集，则从输入层到最上层依次采用 $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ 。高斯丢弃机制中 σ 的值被设定为 $\sqrt{\frac{1-p}{p}}$ 。所有实验结果均为在10种不同的随机种子下取平均值所得。

10. 高斯乘性噪声

丢弃机制的核心做法是将隐藏层激活值与伯努利分布随机变量相乘，这类随机变量以概率 p 取值1，否则取值0。这一思路可以通过用来自其他分布的随机变量来替代前者而得到推广。我们最近发现，使用来自 $\mathcal{N}(1, 1)$ 范围的随机变量进行乘法运算，其效果与使用伯努利噪声相当，甚至可能更好。这种新型的丢弃机制实际上等价于加入一个均值为0、标准差等于对应单元激活值的高斯分布随机变量。也就是说，每个隐藏层激活值 h_i 都会被扰动为 $h_i + h_i r$ ，其中 $r \sim \mathcal{N}(0, 1)$ ；或者等价地变为 $h_i r'$ ，此时 $r' \sim \mathcal{N}(1, 1)$ 。我们还可以将其进一步推广到 $r' \sim \mathcal{N}(1, \sigma^2)$ 的形式，此时 σ 就会成为另一个需要调整的超参数，就像传统（伯努利）丢弃机制中的 p 一样。由于激活值的期望值不会发生变化，因此在测试阶段无需进行权重缩放。

在本文中，我们将丢弃机制定义为这样一种方法：在训练阶段以概率 p 保留各个单元，而在测试阶段则通过将权重乘以 p 这一系数来降低其数值。另一种能够达到相同效果的方法是，在训练阶段将保留下来的激活值乘以 $1/p$ 来放大它们，而在测试阶段则不对权重进行任何修改。只要对每层的学习率及权重初始化进行适当的调整，这两种方法实际上是等价的。

因此，丢弃机制可以被视作将 h_i 与一个伯努利随机变量 r_b 相乘，该随机变量以 $1/p$ 为取值且出现的概率为 p ，其余情况下取值为0。 $E[r_b] = 1$ 和 $Var[r_b] = (1 - p)/p$ 。对于高斯型乘法噪声而言，如果我们设定 $\sigma^2 = (1 - p)/p$ ，那么实际上就是将 h_i 与另一个随机变量 r_g 相乘，其中 $E[r_g] = 1$ 以及 $Var[r_g] = (1 - p)/p$ 。由此可见，这两种形式的丢弃机制都可以被设计成使得被乘的随机变量具有相同的均值和方差。不过，基于这些一阶和二阶矩的特征， r_g 的熵值最高，而 r_b 的熵值最低。虽然这两种极端情况的表现都不错，但表10所示的初步实验结果表明，高熵值的情况可能效果稍好一些。在构建高斯模型时，每一层中的 σ 值都是利用伯努利模型中对应层的 p 来设定为 $\sqrt{\frac{1-p}{p}}$ 的。

11. 结论

丢弃机制是一种通过降低过拟合程度来提升神经网络性能的技术。传统的反向传播学习会形成较为脆弱的协同适应关系，这类关系虽然在训练数据上表现良好，却无法泛化到未见过的数据中。而随机丢弃机制则是通过让某些特定隐藏单元的存在变得不可靠，从而打破这些协同适应关系。

making the presence of any particular hidden unit unreliable. This technique was found to improve the performance of neural nets in a wide variety of application domains including object classification, digit recognition, speech recognition, document classification and analysis of computational biology data. This suggests that dropout is a general technique and is not specific to any domain. Methods that use dropout achieve state-of-the-art results on SVHN, ImageNet, CIFAR-100 and MNIST. Dropout considerably improved the performance of standard neural nets on other data sets as well.

This idea can be extended to Restricted Boltzmann Machines and other graphical models. The central idea of dropout is to take a large model that overfits easily and repeatedly sample and train smaller sub-models from it. RBMs easily fit into this framework. We developed Dropout RBMs and empirically showed that they have certain desirable properties.

One of the drawbacks of dropout is that it increases training time. A dropout network typically takes 2-3 times longer to train than a standard neural network of the same architecture. A major cause of this increase is that the parameter updates are very noisy. Each training case effectively tries to train a different random architecture. Therefore, the gradients that are being computed are not gradients of the final architecture that will be used at test time. Therefore, it is not surprising that training takes a long time. However, it is likely that this stochasticity prevents overfitting. This creates a trade-off between overfitting and training time. With more training time, one can use high dropout and suffer less overfitting. However, one way to obtain some of the benefits of dropout without stochasticity is to marginalize the noise to obtain a regularizer that does the same thing as the dropout procedure, in expectation. We showed that for linear regression this regularizer is a modified form of L2 regularization. For more complicated models, it is not obvious how to obtain an equivalent regularizer. Speeding up dropout is an interesting direction for future work.

Acknowledgments

This research was supported by OGS, NSERC and an Early Researcher Award.

Appendix A. A Practical Guide for Training Dropout Networks

Neural networks are infamous for requiring extensive hyperparameter tuning. Dropout networks are no exception. In this section, we describe heuristics that might be useful for applying dropout.

A.1 Network Size

It is to be expected that dropping units will reduce the capacity of a neural network. If n is the number of hidden units in any layer and p is the probability of retaining a unit, then instead of n hidden units, only pn units will be present after dropout, in expectation. Moreover, this set of pn units will be different each time and the units are not allowed to build co-adaptations freely. Therefore, if an n -sized layer is optimal for a standard neural net on any given task, a good dropout net should have at least n/p units. We found this to be a useful heuristic for setting the number of hidden units in both convolutional and fully connected networks.

实践表明，该技术能够显著提升神经网络在多种应用领域的性能，包括物体分类、数字识别、语音识别、文档分类以及计算生物学数据的分析等。这充分说明丢弃机制是一种通用型技术，并非仅适用于某一特定领域。借助丢弃机制的方法在SVHN、ImageNet、CIFAR-100和MNIST等数据集上均取得了业界领先的结果，同时它也大幅提升了标准神经网络在其他数据集上的表现水平。

这一思路还可以应用于受限玻尔兹曼机以及其他图模型。Dropout的核心思想是，针对那些容易过拟合的大型模型，不断从中抽取并训练较小的子模型。受限玻尔兹曼机很适合纳入这一框架，我们据此开发了带有Dropout机制的受限玻尔兹曼机，并通过实验证明这类模型具备一些理想的特性。

Dropout的一个缺点就是它会增加训练时间。与结构相同的标准神经网络相比，带丢弃机制的神经网络通常需要2到3倍更长的训练时间。造成这一时间增长的主要原因是参数更新过程中存在大量噪声，每个训练案例实际上都在尝试训练不同的随机架构，因此计算得到的梯度并非测试阶段会使用的最终架构的梯度，所以训练耗时较长也就不足为奇了。不过，这种随机性很可能有助于防止过拟合，这就形成了过拟合程度与训练时间之间的权衡。如果给予更多训练时间，就可以使用更高的丢弃比例，从而降低过拟合现象。不过，另一种无需依赖随机性的方法是通过将噪声边缘化，得到一种在期望意义上能实现与Dropout机制相同作用的正则化手段。我们发现，对于线性回归而言，这种正则化方式其实就是L2正则化的改进形式；而对于更为复杂的模型，目前尚不清楚该如何找到具有同等效果的对应正则化方法。加快Dropout的训练速度是未来研究的一个有趣方向。

致谢

本研究得到了OGS机构、NSERC机构以及青年研究者奖的支持。

附录A：训练Dropout网络的实用指南

神经网络因其需要大量的超参数调优而广受诟病，带丢弃机制的神经网络也不例外。在本节中，我们将介绍一些在应用丢弃机制时可能很有用的方法论。

A.1 网络规模

可以预见，启用丢弃机制会降低神经网络的容量。若 n 表示任意层中的隐藏单元数量， p 表示单元的保留概率，那么在应用丢弃机制后，预期内原本的 n 个隐藏单元将只剩下 pn 个。此外，每次生成的这组 pn 个单元都会不同，且这些单元无法自由地进行协同适应。因此，如果对于某项特定任务而言， n 个单元规模的层是标准神经网络的最优选择，那么一个合格的带丢弃机制的神经网络至少应拥有 n/p 个单元。我们发现，这一方法对于确定卷积神经网络和全连接神经网络中隐藏单元的数量而言，是一种非常实用的经验法则。

A.2 Learning Rate and Momentum

Dropout introduces a significant amount of noise in the gradients compared to standard stochastic gradient descent. Therefore, a lot of gradients tend to cancel each other. In order to make up for this, a dropout net should typically use 10-100 times the learning rate that was optimal for a standard neural net. Another way to reduce the effect the noise is to use a high momentum. While momentum values of 0.9 are common for standard nets, with dropout we found that values around 0.95 to 0.99 work quite a lot better. Using high learning rate and/or momentum significantly speed up learning.

A.3 Max-norm Regularization

Though large momentum and learning rate speed up learning, they sometimes cause the network weights to grow very large. To prevent this, we can use max-norm regularization. This constrains the norm of the vector of incoming weights at each hidden unit to be bound by a constant c . Typical values of c range from 3 to 4.

A.4 Dropout Rate

Dropout introduces an extra hyperparameter—the probability of retaining a unit p . This hyperparameter controls the intensity of dropout. $p = 1$, implies no dropout and low values of p mean more dropout. Typical values of p for hidden units are in the range 0.5 to 0.8. For input layers, the choice depends on the kind of input. For real-valued inputs (image patches or speech frames), a typical value is 0.8. For hidden layers, the choice of p is coupled with the choice of number of hidden units n . Smaller p requires big n which slows down the training and leads to underfitting. Large p may not produce enough dropout to prevent overfitting.

Appendix B. Detailed Description of Experiments and Data Sets

This section describes the network architectures and training details for the experimental results reported in this paper. The code for reproducing these results can be obtained from <http://www.cs.toronto.edu/~nitish/dropout>. The implementation is GPU-based. We used the excellent CUDA libraries—cudamat (Mnih, 2009) and cuda-convnet (Krizhevsky et al., 2012) to implement our networks.

B.1 MNIST

The MNIST data set consists of 60,000 training and 10,000 test examples each representing a 28×28 digit image. We held out 10,000 random training images for validation. Hyperparameters were tuned on the validation set such that the best validation error was produced after 1 million weight updates. The validation set was then combined with the training set and training was done for 1 million weight updates. This net was used to evaluate the performance on the test set. This way of using the validation set was chosen because we found that it was easy to set up hyperparameters so that early stopping was not required at all. Therefore, once the hyperparameters were fixed, it made sense to combine the validation and training sets and train for a very long time.

A.2 学习率与动量

与标准的随机梯度下降法相比，丢弃机制会在梯度中引入大量噪声，从而导致许多梯度相互抵消。为了解决这一问题，带丢弃机制的神经网络通常需要使用比标准神经网络最优学习率高10到100倍的参数值。另一种减轻噪声影响的办法是采用较高的动量值。虽然标准神经网络常用的动量值为0.9，但我们在使用丢弃机制时发现，将动量值设定在0.95到0.99之间效果要好得多。使用较高的学习率或动量值能够显著加快模型的学习速度。

A.3 最大范数正则化

虽然较高的动量和学习率能够加快学习速度，但有时也会导致网络权重变得过大。为避免这种情况，我们可以采用最大范数正则化方法，该方法通过将每个隐藏层中各个权重向量的范数限制在某个常数 c 的范围内来进行约束。 c 的典型取值在3到4之间。

A.4 丢弃率

丢弃机制引入了一个额外的超参数——即单元保留概率 p 。这个超参数决定了丢弃机制的强度：当 $p = 1$ 时表示不使用丢弃机制，而较小的 p 值则意味着会应用更强的丢弃效果。隐藏层单元的 p 典型取值范围为0.5到0.8。对于输入层，其取值取决于输入数据的类型；对于实值输入（如图像块或语音帧），典型值为0.8。在隐藏层中， p 的设定通常需要与隐藏单元的数量 n 相结合来考虑。过小的 p 值会要求较大的 n 值，这不仅会降低训练速度，还可能导致模型过拟合；而过大的 p 值则可能无法产生足够的丢弃效果，进而无法防止过拟合现象的出现。

附录B：实验与数据集的详细说明

本节介绍了本文所报告的实验结果所对应的网络架构及训练细节。用于复现这些结果的代码可从<http://www.cs.toronto.edu/~nitish/dropout>获取，该实现基于GPU。我们在构建网络时使用了优秀的CUDA相关库——cudamat (Mnih, 2009) 和 cuda-convnet (克里兹夫斯基等人, 2012)。

B.1 MNIST数据集

MNIST数据集包含60,000个训练样本和10,000个测试样本，每个样本都代表一张 28×28 数字图像。我们另抽出10,000张随机训练图像用于验证。通过在验证集上调整超参数，最终在进行了100万次权重更新后获得了最低的验证误差。之后，该验证集与训练集合并，继续进行总计100万次权重更新的训练。使用这个训练好的网络来评估在测试集上的性能。选择这种方式使用验证集是因为我们发现通过合理设置超参数，完全无需启用提前停止机制；因此一旦超参数确定，将验证集与训练集合并并长期训练是合乎逻辑的。

The architectures shown in Figure 4 include all combinations of 2, 3, and 4 layer networks with 1024 and 2048 units in each layer. Thus, there are six architectures in all. For all the architectures (including the ones reported in Table 2), we used $p = 0.5$ in all hidden layers and $p = 0.8$ in the input layer. A final momentum of 0.95 and weight constraints with $c = 2$ was used in all the layers.

To test the limits of dropout’s regularization power, we also experimented with 2 and 3 layer nets having 4096 and 8192 units. 2 layer nets gave improvements as shown in Table 2. However, the three layer nets performed slightly worse than 2 layer ones with the same level of dropout. When we increased dropout, performance improved but not enough to outperform the 2 layer nets.

B.2 SVHN

The SVHN data set consists of approximately 600,000 training images and 26,000 test images. The training set consists of two parts—A standard labeled training set and another set of labeled examples that are easy. A validation set was constructed by taking examples from both the parts. Two-thirds of it were taken from the standard set (400 per class) and one-third from the extra set (200 per class), a total of 6000 samples. This same process is used by Sermanet et al. (2012). The inputs were RGB pixels normalized to have zero mean and unit variance. Other preprocessing techniques such as global or local contrast normalization or ZCA whitening did not give any noticeable improvements.

The best architecture that we found uses three convolutional layers each followed by a max-pooling layer. The convolutional layers have 96, 128 and 256 filters respectively. Each convolutional layer has a 5×5 receptive field applied with a stride of 1 pixel. Each max pooling layer pools 3×3 regions at strides of 2 pixels. The convolutional layers are followed by two fully connected hidden layers having 2048 units each. All units use the rectified linear activation function. Dropout was applied to all the layers of the network with the probability of retaining the unit being $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ for the different layers of the network (going from input to convolutional layers to fully connected layers). In addition, the max-norm constraint with $c = 4$ was used for all the weights. A momentum of 0.95 was used in all the layers. These hyperparameters were tuned using a validation set. Since the training set was quite large, we did not combine the validation set with the training set for final training. We reported test error of the model that had smallest validation error.

B.3 CIFAR-10 and CIFAR-100

The CIFAR-10 and CIFAR-100 data sets consists of 50,000 training and 10,000 test images each. They have 10 and 100 image categories respectively. These are 32×32 color images. We used 5,000 of the training images for validation. We followed the procedure similar to MNIST, where we found the best hyperparameters using the validation set and then combined it with the training set. The images were preprocessed by doing global contrast normalization in each color channel followed by ZCA whitening. Global contrast normalization means that for image and each color channel in that image, we compute the mean of the pixel intensities and subtract it from the channel. ZCA whitening means that we mean center the data, rotate it onto its principle components, normalize each component

图4中展示的网络架构涵盖了所有由2层、3层或4层构成的网络组合，每层的单元数分别为1024和2048个，总计共有六种不同的架构。对于所有这些架构（包括表2中列出的那些），我们在每一个隐藏层中都使用了 $p = 0.5$ ，而在输入层则使用了 $p = 0.8$ 。此外，所有层都采用了最终动量为0.95的设置，并且施加了 $c = 2$ 的权重约束。

为测试丢弃机制的正则化能力极限，我们还对包含4096个及8192个单元的2层与3层神经网络进行了实验。如表2所示，2层网络确实带来了性能提升，但在相同丢弃率下，3层网络的性能略逊于2层网络。当增加丢弃率时，性能虽有所提升，但仍不足以超越2层网络的表现。

B.2 SVHN

SVHN数据集包含约60万张训练图像和2.6万张测试图像。训练集由两部分组成：一部分是标准的带标签训练样本，另一部分则是较为简单的带标签样本。验证集则是从这两部分样本中选取而来，其中三分之二是来自标准集（每类400张），三分之一来自额外那部分样本（每类200张），总计6000个样本。Sermanet等人（2012）也采用了相同的处理方式。这些输入图像均为经过归一化的RGB像素，其均值设为零，方差为单位方差。而其他预处理方法，诸如全局或局部对比度归一化以及ZCA白化处理，则并未带来显著的性能提升。

我们找到的最佳网络架构包含三层卷积层，每层之后都接有一个最大池化层。这三层卷积层的滤波器数量分别为96、128和256个。每层卷积层都采用 5×5 感受野，并以1像素的步长进行运算；而每个最大池化层则以2像素的步长对 3×3 个区域进行池化处理。在卷积层之后是两层全连接隐藏层，每层包含2048个单元，所有单元均使用修正线性激活函数。对于从输入层到卷积层再到全连接层的不同层次，该网络的所有层都应用了丢弃机制，各层的单元保留概率分别为 $p = (0.9, 0.75, 0.75, 0.5, 0.5, 0.5)$ 。此外，所有权重还受到了 $c = 4$ 的最大范数约束，各层的动量值则为0.95。这些超参数是通过验证集来调整优化的。由于训练集规模较大，我们并未将验证集与训练集合并用于最终训练，而是选择验证误差最小的模型来报告其测试误差。

B.3 CIFAR-10数据集与CIFAR-100数据集

CIFAR-10数据集和CIFAR-100数据集各自包含50,000张训练图像和10,000张测试图像，前者共有10个图像类别，后者则有100个类别。这些都属于 32×32 彩色图像。我们在验证过程中使用了5,000张训练图像。其处理流程与MNIST类似：首先通过验证集找出最优的超参数，再将这些超参数应用到训练集上。在预处理阶段，会对每个颜色通道进行全局对比度归一化处理，随后再执行ZCA白化操作。全局对比度归一化的原理是，针对每张图像及其各个颜色通道，先计算像素强度的平均值，再从该通道的数值中减去这个平均值。而ZCA白化则是将数据均值化，将其旋转到主成分方向上，进而对各个分量进行归一化处理，最后再将数据旋转回原来的状态。

and then rotate it back. The network architecture and dropout rates are same as that for SVHN, except the learning rates for the input layer which had to be set to smaller values.

B.4 TIMIT

The open source Kaldi toolkit (Povey et al., 2011) was used to preprocess the data into log-filter banks. A monophone system was trained to do a forced alignment and to get labels for speech frames. Dropout neural networks were trained on windows of 21 consecutive frames to predict the label of the central frame. No speaker dependent operations were performed. The inputs were mean centered and normalized to have unit variance.

We used probability of retention $p = 0.8$ in the input layers and 0.5 in the hidden layers. Max-norm constraint with $c = 4$ was used in all the layers. A momentum of 0.95 with a high learning rate of 0.1 was used. The learning rate was decayed as $\epsilon_0(1 + t/T)^{-1}$. For DBN pretraining, we trained RBMs using CD-1. The variance of each input unit for the Gaussian RBM was fixed to 1. For finetuning the DBN with dropout, we found that in order to get the best results it was important to use a smaller learning rate (about 0.01). Adding max-norm constraints did not give any improvements.

B.5 Reuters

The Reuters RCV1 corpus contains more than 800,000 documents categorized into 103 classes. These classes are arranged in a tree hierarchy. We created a subset of this data set consisting of 402,738 articles and a vocabulary of 2000 words comprising of 50 categories in which each document belongs to exactly one class. The data was split into equal sized training and test sets. We tried many network architectures and found that dropout gave improvements in classification accuracy over all of them. However, the improvement was not as significant as that for the image and speech data sets. This might be explained by the fact that this data set is quite big (more than 200,000 training examples) and overfitting is not a very serious problem.

B.6 Alternative Splicing

The alternative splicing data set consists of data for 3665 cassette exons, 1014 RNA features and 4 tissue types derived from 27 mouse tissues. For each input, the target consists of 4 softmax units (one for tissue type). Each softmax unit has 3 states (*inc*, *exc*, *nc*) which are of the biological importance. For each softmax unit, the aim is to predict a distribution over these 3 states that matches the observed distribution from wet lab experiments as closely as possible. The evaluation metric is Code Quality which is defined as

$$\sum_{i=1}^{|\text{data points}|} \sum_{t \in \text{tissue types}} \sum_{s \in \{\text{inc}, \text{exc}, \text{nc}\}} p_{i,t}^s \log\left(\frac{q_t^s(r_i)}{\bar{p}^s}\right),$$

where, $p_{i,t}^s$ is the target probability for state s and tissue type t in input i ; $q_t^s(r_i)$ is the predicted probability for state s in tissue type t for input r_i and $\bar{p}^s$ is the average of $p_{i,t}^s$ over i and t .

A two layer dropout network with 1024 units in each layer was trained on this data set. A value of $p = 0.5$ was used for the hidden layer and $p = 0.7$ for the input layer. Max-norm regularization with high decaying learning rates was used. Results were averaged across the same 5 folds used by Xiong et al. (2011).

这两类数据集的网络架构以及丢弃率都与SVHN相同，唯一的不同在于输入层的学习率需要设定得更小一些。

B.4 TIMIT

本研究采用了开源的Kaldi工具包（Povey等人，2011年）对数据进行处理，将其转换为对数滤波器组形式。随后构建了一个单音素系统，用于执行强制对齐并为各语音帧标注标签。接着，使用丢弃神经网络在连续21帧的窗口上对数据进行训练，以此预测窗口中心帧的标签。整个处理过程中未进行任何与说话人特征相关的操作，所有输入数据均经过均值归一化处理，使其单位方差为1。

我们在输入层设置了 $p = 0.8$ 的保留概率，而在隐藏层则设置了0.5的保留概率。所有层的训练均采用了 $c = 4$ 的最大范数约束。此外，还设置了学习率为0.1且动量为0.95的参数组合，该学习率会按照 $\epsilon_0(1 + t/T)^{-1}$ 的规则逐渐衰减。在进行DBN预训练时，我们借助CD-1算法训练了受限玻尔兹曼机；对于高斯RBM模型，每个输入单元的方差被固定为1。在后续对带丢弃机制的DBN进行微调时，我们发现采用较低的学习率（约0.01）才能获得最佳效果，而添加最大范数约束并未带来任何改善。

B.5 Reuters

Reuters RCV1语料库包含超过80万份文档，这些文档被划分到103个类别中，且这些类别按照树形层次结构排列。我们从该数据集中提取出了一个子集，其中包含402,738篇文章以及由50个类别构成的2000个词汇表，每篇文档都属于这50个类别中的一个。随后，这些数据被均等划分为训练集和测试集。我们尝试了多种网络架构，结果发现丢弃机制在提升分类精度方面优于其他所有架构，不过其提升效果并未像在图像和语音数据集上那样显著。这可能是因为该数据集规模相当庞大（训练样本超过20万），因此过拟合并非十分严重的问题。

B.6 可变剪接

该可变剪接数据集包含了来自27种小鼠组织的3665个盒式外显子、1014种RNA特征以及4种组织类型的相关数据。对于每一组输入数据，对应的输出包含4个softmax单元（每个单元对应一种组织类型）。每个softmax单元都有3种状态（*inc*、*exc*、*nc*），而这三种状态在生物学上具有重要意义。针对每一个softmax单元，我们的目标是预测出与实验室实验观测到的分布尽可能匹配的这3种状态的概率分布。评估指标为代码质量指标，其定义如下：

$$\sum_{i=1}^{|\text{data points}|} \sum_{t \in \text{tissue types}} \sum_{s \in \{\text{inc}, \text{exc}, \text{nc}\}} p_{i,t}^s \log\left(\frac{q_t^s(r_i)}{\bar{p}^s}\right),$$

其中， $p_{i,t}^s$ 表示输入 i 中状态 s 与组织类型 t 的目标概率； $q_t^s(r_i)$ 是输入 r_i 在组织类型 t 下状态 s 的预测概率，而 $\bar{p}^s$ 则是 $p_{i,t}^s$ 在 i 和 t 上的平均值。

在该数据集上训练了一个包含两层的Dropout网络，每层各有1024个单元。其中隐藏层采用了 $p = 0.5$ 作为丢弃概率，输入层则采用 $p = 0.7$ 。训练过程中运用了最大范数正则化方法，并设置了逐渐衰减的学习率。最终结果是根据熊等人（2011年）所使用的相同5折交叉验证方式取平均值得到的。

References

- M. Chen, Z. Xu, K. Weinberger, and F. Sha. Marginalized denoising autoencoders for domain adaptation. In *Proceedings of the 29th International Conference on Machine Learning*, pages 767–774. ACM, 2012.
- G. E. Dahl, M. Ranzato, A. Mohamed, and G. E. Hinton. Phone recognition with the mean-covariance restricted Boltzmann machine. In *Advances in Neural Information Processing Systems 23*, pages 469–477, 2010.
- O. Dekel, O. Shamir, and L. Xiao. Learning to classify with missing and corrupted features. *Machine Learning*, 81(2):149–178, 2010.
- A. Globerson and S. Roweis. Nightmare at test time: robust learning by feature deletion. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 353–360. ACM, 2006.
- I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. In *Proceedings of the 30th International Conference on Machine Learning*, pages 1319–1327. ACM, 2013.
- G. Hinton and R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504 – 507, 2006.
- G. E. Hinton, S. Osindero, and Y. Teh. A fast learning algorithm for deep belief nets. *Neural Computation*, 18:1527–1554, 2006.
- K. Jarrett, K. Kavukcuoglu, M. Ranzato, and Y. LeCun. What is the best multi-stage architecture for object recognition? In *Proceedings of the International Conference on Computer Vision (ICCV’09)*. IEEE, 2009.
- A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems 25*, pages 1106–1114, 2012.
- Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- Y. Lin, F. Lv, S. Zhu, M. Yang, T. Cour, K. Yu, L. Cao, Z. Li, M.-H. Tsai, X. Zhou, T. Huang, and T. Zhang. Imagenet classification: fast descriptor coding and large-scale svm training. Large scale visual recognition challenge, 2010.
- A. Livnat, C. Papadimitriou, N. Pippenger, and M. W. Feldman. Sex, mixability, and modularity. *Proceedings of the National Academy of Sciences*, 107(4):1452–1457, 2010.
- V. Mnih. CUDAMat: a CUDA-based matrix class for Python. Technical Report UTML TR 2009-004, Department of Computer Science, University of Toronto, November 2009.

参考文献

- M. Chen、Z. Xu、K. Weinberger和F. Sha撰文的《用于领域适应的边际化去噪自编码器》发表在第29届国际机器学习大会论文集中，页码为767–774，美国计算机协会出版，时间为2012年。G. E. Dahl、M. Ranzato、A. Mohamed和G. E. Hinton撰文的《基于均值-协方差受限玻尔兹曼机的电话识别技术》收录于《神经信息处理系统进展》第23卷，页码为469–477，出版于2010年。O. Dekel、O. Shamir和L. Xiao撰文的《在特征缺失或损坏情况下进行分类的学习方法》发表于《机器学习》杂志，第81卷第2期，页码为149–178，时间为2010年。A. Globerson和S. Roweis撰文的《测试阶段的难题：通过特征删除实现稳健学习》刊登在第23届国际机器学习大会论文集中，页码为353–360，美国计算机协会出版，时间为2006年。I. J. Goodfellow、D. Warde-Farley、M. Mirza、A. Courville和Y. Bengio撰文的《Maxout网络》发表在第30届国际机器学习大会论文集上，页码为1319–1327，美国计算机协会出版，时间为2013年。G. Hinton和R. Salakhutdinov撰文的《利用神经网络降低数据维度》刊登在《科学》杂志上，第313卷第5786期，页码为504–507，时间为2006年。G. E. Hinton、S. Osindero和Y. Teh撰文的《一种用于深度信念网络的快速学习算法》发表在《神经计算》杂志上，第18卷，页码为1527–1554，时间为2006年。K. Jarrett、K. Kavukcuoglu、M. Ranzato和Y. LeCun撰文的《物体识别的最佳多阶段网络架构是什么？》收录在ICCV’09国际计算机视觉大会论文集中，电气和电子工程师协会出版，时间为2009年。A. Krizhevsky撰写的《从极小尺寸图像中学习多层特征》是一份技术报告，由多伦多大学于2009年发布。A. Krizhevsky、I. Sutskever和G. E. Hinton撰文的《利用深度卷积神经网络进行ImageNet分类》发表在《神经信息处理系统进展》第25卷上，页码为1106–1114，时间为2012年。Y. LeCun、B. Boser、J. S. Denker、D. Henderson、R. E. Howard、W. Hubbard和L. D. Jackel撰文的《将反向传播应用于手写邮政编码识别》刊登在《神经计算》杂志上，第1卷第4期，页码为541–551，时间为1989年。Y. Lin、F. Lv、S. Zhu、M. Yang、T. Cour、K. Yu、L. Cao、Z. Li、M.-H. Tsai、X. Zhou、T. Huang和T. Zhang撰文的《ImageNet分类：快速描述符编码与大规模支持向量机训练》发表于2010年的大规模视觉识别挑战赛中。A. Livnat、C. Papadimitriou、N. Pippenger和M. W. Feldman撰文的《性别、可混合性与模块化》发表在《美国国家科学院院刊》上，第107卷第4期，页码为1452–1457，时间为2010年。V. Mnih撰写的《CUDAMat：一种基于CUDA的Python矩阵类》是一份技术报告，编号为UTML TR 2009-004，由多伦多大学计算机科学系于2009年11月发布。

- A. Mohamed, G. E. Dahl, and G. E. Hinton. Acoustic modeling using deep belief networks. *IEEE Transactions on Audio, Speech, and Language Processing*, 2010.
- R. M. Neal. *Bayesian Learning for Neural Networks*. Springer-Verlag New York, Inc., 1996.
- Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011.
- S. J. Nowlan and G. E. Hinton. Simplifying neural networks by soft weight-sharing. *Neural Computation*, 4(4), 1992.
- D. Povey, A. Ghoshal, G. Boulianne, L. Burget, O. Glembek, N. Goel, M. Hannemann, P. Motlicek, Y. Qian, P. Schwarz, J. Silovsky, G. Stemmer, and K. Vesely. The Kaldi Speech Recognition Toolkit. In *IEEE 2011 Workshop on Automatic Speech Recognition and Understanding*. IEEE Signal Processing Society, 2011.
- R. Salakhutdinov and G. Hinton. Deep Boltzmann machines. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, volume 5, pages 448–455, 2009.
- R. Salakhutdinov and A. Mnih. Bayesian probabilistic matrix factorization using Markov chain Monte Carlo. In *Proceedings of the 25th International Conference on Machine Learning*. ACM, 2008.
- J. Sanchez and F. Perronnin. High-dimensional signature compression for large-scale image classification. In *Proceedings of the 2011 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1665–1672, 2011.
- P. Sermanet, S. Chintala, and Y. LeCun. Convolutional neural networks applied to house numbers digit classification. In *International Conference on Pattern Recognition (ICPR 2012)*, 2012.
- P. Simard, D. Steinkraus, and J. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *Proceedings of the Seventh International Conference on Document Analysis and Recognition*, volume 2, pages 958–962, 2003.
- J. Snoek, H. Larochelle, and R. Adams. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems 25*, pages 2960–2968, 2012.
- N. Srebro and A. Shraibman. Rank, trace-norm and max-norm. In *Proceedings of the 18th annual conference on Learning Theory, COLT’05*, pages 545–560. Springer-Verlag, 2005.
- N. Srivastava. Improving Neural Networks with Dropout. Master’s thesis, University of Toronto, January 2013.
- R. Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B. Methodological*, 58(1):267–288, 1996.

A. Mohamed、G. E. Dahl以及G. E. Hinton共同发表了题为《利用深度信念网络进行声学建模》的文章，该文发表于2010年的《*IEEE*音频、语音与语言处理汇刊》。R. M. Neal则撰写了《《神经网络的贝叶斯学习》》一书，由Springer-Verlag New York, Inc.于1996年出版。Y. Netzer等人通过《2011年NIPS深度学习与无监督特征学习研讨会》发表了关于利用无监督特征学习在自然图像中识别数字的研究成果。S. J. Nowlan与G. E. Hinton在1992年的《神经计算》第4卷第4期中提出了通过软权重共享简化神经网络的方法。D. Povey等人为《IEEE 2011年自动语音识别与理解研讨会》编写了《Kaldi语音识别工具包》，该工具包收录于IEEE信号处理学会2011年出版的相关会议论文集中。R. Salakhutdinov与G. Hinton在2009年的《人工智能与统计国际会议论文集》第5卷中阐述了深度玻尔兹曼机的原理。随后他们又在2008年的ACM举办的《第25届国际机器学习会议》论文集中，借助马尔可夫链蒙特卡洛方法介绍了基于贝叶斯的概率矩阵分解技术。J. Sanchez与F. Perronnin在2011年的《2011年IEEE计算机视觉与模式识别会议》论文集中探讨了用于大规模图像分类的高维签名压缩方法。P. Sermanet等人将卷积神经网络应用于门牌号数字识别研究，并将该成果发表在2012年的《2012年国际模式识别会议（ICPR 2012）》论文集中。P. Simard、D. Steinkraus与J. Platt在2003年的《第七届文档分析与识别国际会议》论文集第2卷中总结了将卷积神经网络用于视觉文档分析的最佳实践。J. Snoek、H. Larochelle与R. Adams在2012年的《《神经信息处理系统进展》第25卷》中介绍了机器学习算法的实用贝叶斯优化方法。N. Srebro与A. Shraibman则在2005年的COLT’05年第18届年度学习理论会议相关论文中论述了秩、迹范数以及最大范数正则化等内容。N. Srivastava在2013年1月的多伦多大学完成的硕士论文中，研究了如何通过丢弃机制改进神经网络性能。最后，R. Tibshirani在1996年的《皇家统计学会B系列：方法论杂志》第58卷第1期中发表了关于通过套索算法实现回归收缩与选择的相关研究。

A. N. Tikhonov. On the stability of inverse problems. *Doklady Akademii Nauk SSSR*, 39(5): 195–198, 1943.

L. van der Maaten, M. Chen, S. Tyree, and K. Q. Weinberger. Learning with marginalized corrupted features. In *Proceedings of the 30th International Conference on Machine Learning*, pages 410–418. ACM, 2013.

P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning*, pages 1096–1103. ACM, 2008.

P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. In *Proceedings of the 27th International Conference on Machine Learning*, pages 3371–3408. ACM, 2010.

S. Wager, S. Wang, and P. Liang. Dropout training as adaptive regularization. In *Advances in Neural Information Processing Systems 26*, pages 351–359, 2013.

S. Wang and C. D. Manning. Fast dropout training. In *Proceedings of the 30th International Conference on Machine Learning*, pages 118–126. ACM, 2013.

H. Y. Xiong, Y. Barash, and B. J. Frey. Bayesian prediction of tissue-regulated splicing using RNA sequence and cellular context. *Bioinformatics*, 27(18):2554–2562, 2011.

M. D. Zeiler and R. Fergus. Stochastic pooling for regularization of deep convolutional neural networks. *CoRR*, abs/1301.3557, 2013.

A. N. Tikhonov在1943年的《苏联科学院院报》第39卷第5期中探讨了逆问题稳定性相关议题。L. van der Maaten、M. Chen、S. Tyree与K. Q. Weinberger在2013年的ACM举办的《第30届国际机器学习会议》论文集中发表了关于利用存在噪声且被边缘化的特征进行学习文章。P. Vincent、H. Larochelle、Y. Bengio及P.-A. Manzagol在2008年的ACM举办的《第25届国际机器学习会议》论文集中介绍了如何运用去噪自编码器提取并构建稳健的特征。同年，这组学者还在同一会议的论文集中发表了关于堆叠式去噪自编码器的研究，该技术通过局部去噪准则在深层网络中学习有用的表征。P. Vincent等人随后在2010年的ACM举办的《第27届国际机器学习会议》论文集中进一步阐述了相关技术。S. Wager、S. Wang与P. Liang在2013年的《《神经信息处理系统进展》第26卷》中提出，可将丢弃机制视为一种自适应的正则化手段。S. Wang与C. D. Manning则在2013年的《第30届国际机器学习会议》论文集中展示了快速的丢弃机制训练方法。H. Y. Xiong、Y. Barash与B. J. Frey在2011年的《生物信息学》杂志第27卷第18期中，利用RNA序列与细胞环境信息实现了对组织调控剪接过程的贝叶斯预测。M. D. Zeiler与R. Fergus在2013年的计算机研究快报上发布了题为“用于深度卷积神经网络正则化的随机池化”嘅研究论文，对应编号为abs/1301.3557。